

疯狂地产

Crazy Estate (英文名) • V4.1.0

Source code excerpt (first 30 + last 30 pages)

Scope: Assets_Game own source. Files: 88 / Lines: 26247 / 50 lines per page / Total pages: 525

Generated: 2026-09-27 19:09:55 (local build chain, headless Edge print)

```
// ===== FILE: Assets\_Game\Scripts\Editor\AchieveCheck.cs =====
// G16 AchieveCheck - T012 slice C honor wall gate (AC1-AC12) plus the
// slice D view-face laws (AC13-AC15: wall body compose / count face /
// shine-vs-dim tint - the view statics stay this gate's single truth).
// Pure C#, zero-scene. Eight derive-only honors (requirements doc M07
// section 6): first home / bargain god (best single-duel cut >= ten wan)
// / rent freedom (per-beat rent flow outruns the mortgage bill) /
// review-literature collector / tenant codex / city crown (H10 in the
// ledger) / zero-ad completion (a home paid off installment by
// installment with zero ad grants, U091) / tycoon (the other seven all
// shine). Wall faces compose through CopyTable (title / count / line),
// the zero-ad honor reuses the T010 "zero_ad_achievement" key, the copy
// faces sit on both locale faces with the U129 policy-word scan and a
// live positive control, and a full double drive lands byte-identical
// wall stamps (determinism, zero RNG).
// Report: Logs/g16_achieve_check_report.txt (first line
// ALL_GREEN passed=N failed=M).
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Text;
using System.Text.RegularExpressions;
using UnityEditor;

namespace Sy.CrazyEstate
{
    public static class AchieveCheck
    {
        {
            static readonly List<string> Lines = new List<string>();
            static int _passed;
            static int _failed;

            // U129 banned-word dictionary (script "Golden Twenty Years" section 3b),
            // written as unicode escapes so the source stays pure ASCII (N8 rule).
            const string PolicyWordPattern =
                "\u653F\u7B56|\u65B0\u653F|\u8C03\u63A7|\u68DA\u6539|\u65E7\u6539|\u9650\u8D2D|\u9650\u8D37|\u9650\u552E"
                + "|\u623F\u4EA7\u7A0E|\u6301\u6709\u7A0E|\u5951\u7A0E|\u79DF\u8D41\u5E76\u4E3E|\u4FDD\u4EA4\u697C"
                +
                "|\u623F\u4F4F\u4E0D\u7092|\u4E09\u6761\u7EA2\u7EBF|\u516C\u79EF\u91D1|\u843D\u6237|\u5F81\u6536|\u7ECF\u6D4E\u9002\u7528";

            static readonly string[] NewZhKeys =
            {
                "ach_wall_title", "ach_wall_count", "ach_locked",
                "ach_first_home", "ach_bargain_god", "ach_rent_freedom",
                "ach_review_collector", "ach_tenant_codex", "ach_city_crown",
                "ach_tycoon",
            };
        };

        static void Assert(string id, bool ok, string detail)
        {

```

```

    if (ok) _passed++; else _failed++;
    Lines.Add("[ " + (ok ? "PASS" : "FAIL") + " ] " + id + " - " + detail);
}

public static void Run()
{
    try
    {
        // AC1 anchor law (requirements doc section 6): eight honors
        // with contiguous ids 1..8, canonical ach_NN stems, every
        // copy face resolving on EN (zero-ad reuses the T010 key),
        // tycoon last, wall total frozen at 8.
        var stemsOk = true;
        var facesOk = true;
        for (var id = AchieveId.FirstHome; id <= AchieveId.Tycoon; id++)
        {
            if (AchieveSim.StemOf(id) != "ach " + ((int)id).ToString("D2",
                CultureInfo.InvariantCulture)) stemsOk = false;
            if (!CopyTable.TryGetRaw(AchieveSim.CopyKeyOf(id), out var en)
                || string.IsNullOrEmpty(en)) facesOk = false;
        }
        Assert("AC1_anchor_law",
            stemsOk && facesOk && AchieveSim.Total == 8
            && AchieveSim.ZeroAdCopyKey == "zero_ad_achievement"
            && (int)AchieveId.Tycoon == 8,
            "stems=" + stemsOk + " faces=" + facesOk
            + " total=" + AchieveSim.Total + " zeroAdKey=" + AchieveSim.ZeroAdCopyKey);

        // AC2 first-home + no-bill comedy law: the blank wall reads
        // 00000000; one owned home lights FirstHome AND RentFreedom
        // (no bill = instant freedom), nothing else.
        var blankStamp = AchieveSim.WallStamp(null, null, null, null, 0f);
        var one = new RentLedger();
        one.AddOwned(1);
        var oneStamp = AchieveSim.WallStamp(one, null, null, null, 0f);
        var oneCount = AchieveSim.UnlockedCount(one, null, null, null, 0f);
        Assert("AC2_first_home_law",
            blankStamp == "00000000" && oneStamp == "10100000" && oneCount == 2,
            "blank=" + blankStamp + " oneHome=" + oneStamp + " count=" + oneCount);

        // AC3 bargain-god law: the guided duel on the 40-wan starter
        // home (three correct bubbles 40 -> 35, savings 5 = 12.5% of
        // list, slice B % band) with the deal signed lands
        // BestSavingsWan = 5 >= 5; a fresh bargain
        // without a signed deal stays locked.
        var bargain = new BargainSim();
        bargain.StartBargain(40, true);
        bargain.ResolveBubble(bargain.CorrectTalkId, false);
        bargain.ResolveBubble(bargain.CorrectTalkId, false);
        bargain.ResolveBubble(bargain.CorrectTalkId, false);
    }
}

```

```

var receipt = bargain.TrySignDeal(20);
var freshLocked = AchieveSim.IsUnlocked(AchieveId.BargainGod,
    null, new BargainSim(), null, null, 0f);
Assert("AC3_bargain_god_law",
    receipt.Accepted && receipt.SavingsWan == 5
    && bargain.BestSavingsWan == 5
    && AchieveSim.IsUnlocked(AchieveId.BargainGod, null, bargain, null, null, 0f)
    && !freshLocked,
    "accepted=" + receipt.Accepted + " savings=" + receipt.SavingsWan
    + " best=" + bargain.BestSavingsWan + " freshLocked=" + freshLocked);

// AC4 rent-freedom law: rent flow 60 outruns no bill (unlocked);
// the starter mortgage bills 78 and locks it; two homes flow
// 204 > 78 unlocks it again.
var mortgage = new MortgageSim();
mortgage.AddLoan(1, 40, 12, 100);
var free = AchieveSim.IsUnlocked(AchieveId.RentFreedom,
    one, null, null, null, 0f);
var squeezed = AchieveSim.IsUnlocked(AchieveId.RentFreedom,
    one, null, mortgage, null, 0f);
var two = new RentLedger();
two.AddOwned(1);
two.AddOwned(3);
var flowing = AchieveSim.IsUnlocked(AchieveId.RentFreedom,
    two, null, mortgage, null, 0f);
Assert("AC4_rent_freedom_law",
    free && !squeezed && flowing && mortgage.DueCoins(0f) == 78,
    "noBill=" + free + " squeezed=" + !squeezed
    + " twoHomes=" + flowing + " due=" + mortgage.DueCoins(0f));

// AC5 codex-duo law: ten deterministic seats need ten rows;
// the review-literature face mirrors the tenant face, so a
// full roster lights BOTH collector honors; nine rows keep
// both dark.
var full = new RentLedger();
for (var i = 0; i < 10; i++) full.AddOwned(1);
var nine = new RentLedger();
for (var i = 0; i < 9; i++) nine.AddOwned(1);
var tenantFace = AlbumSim.TenantAlbum(full);
var reviewFace = AlbumSim.ReviewAlbum(full);
var tenantsUnlocked = 0;
for (var i = 0; i < tenantFace.Count; i++) if (tenantFace[i].Unlocked) tenantsUnlocked++;
var fullOk = AchieveSim.IsUnlocked(AchieveId.TenantCodex, full, null, null, null, 0f)
    && AchieveSim.IsUnlocked(AchieveId.ReviewCollector, full, null, null, null, 0f);
var nineLocked = !AchieveSim.IsUnlocked(AchieveId.TenantCodex, nine, null, null, null, 0f)
    && !AchieveSim.IsUnlocked(AchieveId.ReviewCollector, nine, null, null, null, 0f);
Assert("AC5_codex-duo_law",
    tenantsUnlocked == 10 && tenantFace.Count == 10 && reviewFace.Count == 10
    && fullOk && nineLocked,
    "seats=" + tenantsUnlocked + "/" + tenantFace.Count

```

```

    + " full=" + fullOk + " nineLocked=" + nineLocked);

// AC6 city-crown law: the World Crown (H10) in the ledger
// lights the crown (production attainability rides the
// H04-H10 listing expansion batch; the law is unit-id based
// and the unknown-listing fallback keeps the derive safe).
var crown = new RentLedger();
crown.AddOwned(10);
var crownOk = AchieveSim.IsUnlocked(AchieveId.CityCrown, crown, null, null, null, 0f)
    && !AchieveSim.IsUnlocked(AchieveId.CityCrown, one, null, null, null, 0f);
Assert("AC6_city_crown_law", crownOk,
    "h10Owned=" + AchieveSim.IsUnlocked(AchieveId.CityCrown,
        crown, null, null, null, 0f)
    + " h01Only=" + AchieveSim.IsUnlocked(AchieveId.CityCrown,
        one, null, null, null, 0f));

// AC7 zero-ad law (U091): the starter loan paid off
// installment by installment (359 months, MG5 face) with a
// zero-grant IAA row lights the honor; one ad grant darkens
// it again; nothing paid yet keeps it dark.
var paidOff = DrivePayoff(new IaaSim());
var adIaa = new IaaSim();
adIaa.ViewDoubleAd("d1", 0);
var paidWithAd = DrivePayoff(adIaa);
var freshMortgage = new MortgageSim();
freshMortgage.AddLoan(1, 40, 12, 100);
Assert("AC7_zero_ad_law",
    paidOff && !paidWithAd
    && !AchieveSim.IsUnlocked(AchieveId.ZeroAd,
        null, null, freshMortgage, new IaaSim(), 0f),
    "purePayoff=" + paidOff + " oneAd=" + !paidWithAd
    + " unpaidLocked=" + !AchieveSim.IsUnlocked(AchieveId.ZeroAd,
        null, null, freshMortgage, new IaaSim(), 0f));

// AC8 tycoon law: the full drive - eleven rows (ten seats +
// the crown), the 13-wan god cut, the paid-off mortgage, the
// zero-grant IAA row - lights all eight honors at once.
var fullDrive = DriveFull();
var fullStamp = AchieveSim.WallStamp(fullDrive.Ledger, fullDrive.Bargain,
    fullDrive.Mortgage, fullDrive.Iaa, 0f);
var fullCount = AchieveSim.UnlockedCount(fullDrive.Ledger, fullDrive.Bargain,
    fullDrive.Mortgage, fullDrive.Iaa, 0f);
Assert("AC8_tycoon_law",
    fullStamp == "11111111" && fullCount == 8
    && AchieveSim.IsUnlocked(AchieveId.Tycoon, fullDrive.Ledger,
        fullDrive.Bargain, fullDrive.Mortgage, fullDrive.Iaa, 0f),
    "stamp=" + fullStamp + " count=" + fullCount);

// AC9 wall-face law: the count line formats through the copy
// table, the title resolves, locked lines read the locked copy

```

```

// and an unlocked zero-ad line reads the T010 key verbatim.
var countFace = AchieveSim.ComposeWallCount(3, 8);
var countExpected = string.Format(CultureInfo.InvariantCulture,
    CopyTable.Get("ach_wall_count"), 3, 8);
var lockedLine = AchieveSim.ComposeWallLine(new AchieveEntry
{ Id = AchieveId.FirstHome, Stem = "ach_01", CopyKey = "ach_first_home", Unlocked = false });
var zeroAdLine = AchieveSim.ComposeWallLine(new AchieveEntry
{ Id = AchieveId.ZeroAd, Stem = "ach_07", CopyKey = AchieveSim.ZeroAdCopyKey, Unlocked = true });
Assert("AC9_wall_face_law",
    countFace == countExpected && countFace.IndexOf('{') < 0
    && lockedLine == CopyTable.Get("ach_locked")
    && zeroAdLine == CopyTable.Get("zero_ad_achievement")
    && AchieveSim.WallTitle() == CopyTable.Get("ach_wall_title"),
    "count=" + (countFace == countExpected)
    + " locked=" + (lockedLine == CopyTable.Get("ach_locked"))
    + " zeroAd=" + (zeroAdLine == CopyTable.Get("zero_ad_achievement")));

// AC10 copy-face law: the ten new wall keys exist on both
// locale faces, sit on the single-authority key list, and
// the shared validator (coverage + non-empty + placeholder
// parity) stays clean.
var zh = CopyTableZh.Snapshot();
var keyListOk = true;
var faceOk = true;
foreach (var k in NewZhKeys)
{
    if (Array.IndexOf(CopyTable.ZhKeyList, k) < 0) keyListOk = false;
    if (!CopyTable.TryGetRaw(k, out var en) || string.IsNullOrEmpty(en)) faceOk = false;
    if (!zh.TryGetValue(k, out var z) || string.IsNullOrEmpty(z)) faceOk = false;
}
var problems = CopyTable.Validate(new List<string>(), zh);
Assert("AC10_copy_face_law",
    keyListOk && faceOk && problems.Count == 0,
    "keyList=" + keyListOk + " faces=" + faceOk
    + " validatorProblems=" + problems.Count + " keys=" + NewZhKeys.Length);

// AC11 policy-word scan (U129): the new ZH comedy copy stays
// clean of the banned dictionary; the positive control proves
// the scanner is alive.
var bannedHits = 0;
foreach (var k in NewZhKeys)
    if (zh.TryGetValue(k, out var v) && Regex.IsMatch(v, PolicyWordPattern))
        bannedHits++;
var control = Regex.IsMatch("\u8C03\u63A7", PolicyWordPattern);
Assert("AC11_policy_scan_law",
    bannedHits == 0 && control,
    "hits=" + bannedHits + " control=" + control);

// AC12 determinism law: two identical full drives land
// byte-identical wall stamps, counts and composed wall lines.

```

```

var a = DriveFull();
var b = DriveFull();
var stampA = AchieveSim.WallStamp(a.Ledger, a.Bargain, a.Mortgage, a.Iaa, 0f);
var stampB = AchieveSim.WallStamp(b.Ledger, b.Bargain, b.Mortgage, b.Iaa, 0f);
var linesA = WallLinesJoin(a);
var linesB = WallLinesJoin(b);
Assert("AC12_determinism_law",
    stampA == stampB && stampA == "11111111" && linesA == linesB,
    "stampMatch=" + (stampA == stampB) + " linesMatch=" + (linesA == linesB));

// AC13 wall-body law (slice D view face): the host body
// composes one line per honor in id order - the full drive
// reads each honor's own copy face, the all-null boot face
// reads eight locked lines, never a blank crash.
var fullBody = AchieveViewHost.ComposeWallBody(fullDrive.Ledger, fullDrive.Bargain,
    fullDrive.Mortgage, fullDrive.Iaa, 0f);
var blankBody = AchieveViewHost.ComposeWallBody(null, null, null, null, 0f);
var fullLines = fullBody.Split('\n');
var blankLines = blankBody.Split('\n');
var bodyOk = fullLines.Length == AchieveSim.Total && blankLines.Length == AchieveSim.Total;
for (var i = 0; i < AchieveSim.Total && bodyOk; i++)
{
    if (fullLines[i] != CopyTable.Get(AchieveSim.CopyKeyOf((AchieveId)(i + 1)))) bodyOk = false;
    if (blankLines[i] != CopyTable.Get("ach_locked")) bodyOk = false;
}
Assert("AC13_wall_body_law", bodyOk,
    "lines=" + fullLines.Length + "/" + blankLines.Length + " faceOrder=" + bodyOk);

// AC14 count-face law: the wall count composes through the
// copy table (no raw format residue) - blank 0/8, full 8/8.
var blankCountFace = AchieveViewHost.ComposeCountFace(null, null, null, null, 0f);
var fullCountFace = AchieveViewHost.ComposeCountFace(fullDrive.Ledger, fullDrive.Bargain,
    fullDrive.Mortgage, fullDrive.Iaa, 0f);
Assert("AC14_count_face_law",
    blankCountFace == string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("ach_wall_count"), 0, AchieveSim.Total)
    && fullCountFace == string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("ach_wall_count"), AchieveSim.Total, AchieveSim.Total)
    && blankCountFace.IndexOf('{') < 0 && fullCountFace.IndexOf('{') < 0,
    "blank=" + blankCountFace + " full=" + fullCountFace);

// AC15 shine-vs-dim law: earned honors glow gold (warm
// family, r >= g >= b), locked honors sit neutral gray
// (r == g == b) - the wall reads at one glance.
var gold = AchieveViewHost.LineTint(true);
var dim = AchieveViewHost.LineTint(false);
Assert("AC15_shine_dim_law",
    gold.r >= gold.g && gold.g >= gold.b && gold.r > gold.b
    && dim.r == dim.g && dim.g == dim.b,
    "gold=" + gold.r.ToString("0.00", CultureInfo.InvariantCulture)

```

```

+ "/" + gold.g.ToString("0.00", CultureInfo.InvariantCulture)
+ "/" + gold.b.ToString("0.00", CultureInfo.InvariantCulture)
+ " dim=" + dim.r.ToString("0.00", CultureInfo.InvariantCulture));

// AC16 finale-axis law (T020-d five-line archive): each line
// has its own single-axis finale - LIC01 rides RentFreedom
// (two homes flow 204 > bill 78; one home 60 < 78 stays dark),
// LIC02 needs the top holding tier (five keys; four is dark),
// LIC03 needs the full-house star cap (210 wan = five stars;
// 209 is dark), LIC04 rides CityCrown (H10 row; H01-only is
// dark), LIC05 needs three pool gavels (two is dark).
var twoHomes = new RentLedger();
twoHomes.AddOwned(1);
twoHomes.AddOwned(3);
var billMortgage = new MortgageSim();
billMortgage.AddLoan(1, 40, 12, 100);
var squeezedLedger = new RentLedger();
squeezedLedger.AddOwned(1);
var empireFull = new RentLedger();
for (var i = 0; i < AchieveSim.StorageEmpireHoldings; i++) empireFull.AddOwned(1);
var empireShort = new RentLedger();
for (var i = 0; i < AchieveSim.StorageEmpireHoldings - 1; i++) empireShort.AddOwned(1);
var crownLedger = new RentLedger();
crownLedger.AddOwned(10);
var plainLedger = new RentLedger();
plainLedger.AddOwned(1);
var l01Lit = AchieveSim.LineFinaleMet(LicenseId.SmallLandlord,
twoHomes, billMortgage, 0f, 0, 0);
var l01Dark = AchieveSim.LineFinaleMet(LicenseId.SmallLandlord,
squeezedLedger, billMortgage, 0f, 0, 0);
var l02Lit = AchieveSim.LineFinaleMet(LicenseId.RentTycoon,
empireFull, null, 0f, 0, 0);
var l02Dark = AchieveSim.LineFinaleMet(LicenseId.RentTycoon,
empireShort, null, 0f, 0, 0);
var l03Lit = AchieveSim.LineFinaleMet(LicenseId.GoldAgent,
null, null, 0f, StarStoreTable.RungFor(StarStoreTable.MaxStars), 0);
var l03Dark = AchieveSim.LineFinaleMet(LicenseId.GoldAgent,
null, null, 0f, StarStoreTable.RungFor(StarStoreTable.MaxStars) - 1, 0);
var l04Lit = AchieveSim.LineFinaleMet(LicenseId.Developer,
crownLedger, null, 0f, 0, 0);
var l04Dark = AchieveSim.LineFinaleMet(LicenseId.Developer,
plainLedger, null, 0f, 0, 0);
var l05Lit = AchieveSim.LineFinaleMet(LicenseId.BottomFisher,
null, null, 0f, 0, AchieveSim.SnipeKingPoolWins);
var l05Dark = AchieveSim.LineFinaleMet(LicenseId.BottomFisher,
null, null, 0f, 0, AchieveSim.SnipeKingPoolWins - 1);
Assert("AC16_finale_axis_law",
l01Lit && !l01Dark && l02Lit && !l02Dark && l03Lit && !l03Dark
&& l04Lit && !l04Dark && l05Lit && !l05Dark,
"01=" + l01Lit + "/" + !l01Dark + " 02=" + l02Lit + "/" + !l02Dark

```

```

+ " 03=" + l03Lit + "/" + !l03Dark + " 04=" + l04Lit + "/" + !l04Dark
+ " 05=" + l05Lit + "/" + !l05Dark);

// AC17 finale stamp/copy law: the full drive projects "11111"
// with count 5, the blank wall "00000" with count 0; every
// badge key resolves on both locale faces, sits on the
// authority key list with placeholder parity, maps one key
// per line id, and the ZH comedy copy stays clean of the
// U129 dictionary (positive control proves the scanner).
var finFullDrive = DriveFinaleFull();
var finStamp = AchieveSim.FinaleStamp(finFullDrive.Ledger, finFullDrive.Mortgage,
    0f, finFullDrive.StarSavingsWan, finFullDrive.SnipePoolWins);
var finCount = AchieveSim.FinaleLitCount(finFullDrive.Ledger, finFullDrive.Mortgage,
    0f, finFullDrive.StarSavingsWan, finFullDrive.SnipePoolWins);
var finBlank = AchieveSim.FinaleStamp(null, null, 0f, 0, 0);
var finKeyOk = true;
for (var i = 0; i < LicenseTable.Lines.Length; i++)
{
    var k = AchieveSim.FinaleCopyKeyOf(LicenseTable.Lines[i].Id);
    if (k != "lic_finale_0" + (i + 1)) finKeyOk = false;
    if (Array.IndexOf(CopyTable.ZhKeyList, k) < 0) finKeyOk = false;
    if (!CopyTable.TryGetRaw(k, out var finEn) || string.IsNullOrEmpty(finEn)) finKeyOk = false;
    if (!zh.TryGetValue(k, out var finZh) || string.IsNullOrEmpty(finZh)) finKeyOk = false;
    if (CopyTable.PlaceholderCount(finEn) != CopyTable.PlaceholderCount(finZh)) finKeyOk = false;
}
var showKeysOk = true;
var showKeys = new[] { "lic_finale_title", "lic_finale_face",
    "lic_finale_show_banner", "lic_finale_show_speech", "lic_finale_show_crowd" };
for (var i = 0; i < showKeys.Length; i++)
{
    if (Array.IndexOf(CopyTable.ZhKeyList, showKeys[i]) < 0) showKeysOk = false;
    if (!CopyTable.TryGetRaw(showKeys[i], out var sEn) || string.IsNullOrEmpty(sEn)) showKeysOk = false;
    if (!zh.TryGetValue(showKeys[i], out var sZh) || string.IsNullOrEmpty(sZh)) showKeysOk = false;
    if (CopyTable.PlaceholderCount(sEn) != CopyTable.PlaceholderCount(sZh)) showKeysOk = false;
}
var finPolicyHits = 0;
for (var i = 0; i < LicenseTable.Lines.Length; i++)
{
    var k = AchieveSim.FinaleCopyKeyOf(LicenseTable.Lines[i].Id);
    if (zh.TryGetValue(k, out var v) && Regex.IsMatch(v, PolicyWordPattern)) finPolicyHits++;
}
for (var i = 0; i < showKeys.Length; i++)
    if (zh.TryGetValue(showKeys[i], out var v2) && Regex.IsMatch(v2, PolicyWordPattern)) finPolicyHits++;
var finControl = Regex.IsMatch("\u8C03\u63A7", PolicyWordPattern);
Assert("AC17_finale_stamp_copy_law",
    finStamp == "11111" && finCount == 5 && finBlank == "00000"
    && finKeyOk && showKeysOk && finPolicyHits == 0 && finControl
    && AchieveSim.FinaleLineTotal == 5,
    "stamp=" + finStamp + " count=" + finCount + " blank=" + finBlank
    + " keys=" + finKeyOk + " showKeys=" + showKeysOk

```

```

    + " policyHits=" + finPolicyHits + " control=" + finControl);

    // AC18 finale determinism law: two identical full drives land
    // byte-identical stamps, counts and composed badge bodies.
    var finA = DriveFinaleFull();
    var finB = DriveFinaleFull();
    var finStampA = AchieveSim.FinaleStamp(finA.Ledger, finA.Mortgage,
    0f, finA.StarSavingsWan, finA.SnipePoolWins);
    var finStampB = AchieveSim.FinaleStamp(finB.Ledger, finB.Mortgage,
    0f, finB.StarSavingsWan, finB.SnipePoolWins);
    var finBodyA = FinaleViewHost.ComposeBadgeBody(finA.Ledger, finA.Mortgage,
    0f, finA.StarSavingsWan, finA.SnipePoolWins);
    var finBodyB = FinaleViewHost.ComposeBadgeBody(finB.Ledger, finB.Mortgage,
    0f, finB.StarSavingsWan, finB.SnipePoolWins);
    Assert("AC18_finale_determinism_law",
    finStampA == finStampB && finStampA == "11111" && finBodyA == finBodyB
    && finBodyA.IndexOf('{') < 0,
    "stampMatch=" + (finStampA == finStampB) + " bodyMatch=" + (finBodyA == finBodyB));
}
catch (Exception e)
{
    _failed++;
    Lines.Add("[FAIL] AC0_exception - " + e.Message);
}

WriteReport();
}

// The starter loan walked to zero, month by month (359 settles, the
// MGS amortization face), reading back only the zero-ad honor.
static bool DrivePayoff(IaaSim iaa)
{
    var m = new MortgageSim();
    m.AddLoan(1, 40, 12, 100);
    var guard = 0;
    while (m.TotalLoansCoins > 0 && guard++ < 500) m.SettleMonth(78, 100, 0f);
    return AchieveSim.IsUnlocked(AchieveId.ZeroAd, null, null, m, iaa, 0f);
}

sealed class FullDrive
{
    public RentLedger Ledger;
    public BargainSim Bargain;
    public MortgageSim Mortgage;
    public IaaSim Iaa;
}

// T020-d finale full drive: five ledger rows (the World Crown H10
// doubles as the fifth storage-empire key), the starter bill (rent
// flow 906 dwarfs 78), the full-house star rung and three pool

```

```

// gavels - all five finales lit at once.
sealed class FinaleDrive
{
    public RentLedger Ledger;
    public MortgageSim Mortgage;
    public int StarSavingsWan;
    public int SnipePoolWins;
}

static FinaleDrive DriveFinaleFull()
{
    var ledger = new RentLedger();
    ledger.AddOwned(10); // the crown row is also the fifth empire key
    for (var i = 0; i < 4; i++) ledger.AddOwned(1);
    var mortgage = new MortgageSim();
    mortgage.AddLoan(1, 40, 12, 100);
    return new FinaleDrive
    {
        Ledger = ledger,
        Mortgage = mortgage,
        StarSavingsWan = StarStoreTable.RungFor(StarStoreTable.MaxStars),
        SnipePoolWins = AchieveSim.SnipeKingPoolWins,
    };
}

static FullDrive DriveFull()
{
    var ledger = new RentLedger();
    for (var i = 0; i < 10; i++) ledger.AddOwned(1);
    ledger.AddOwned(10); // the World Crown row (eleventh)
    var bargain = new BargainSim();
    bargain.StartBargain(40, true);
    bargain.ResolveBubble(bargain.CorrectTalkId, false);
    bargain.ResolveBubble(bargain.CorrectTalkId, false);
    bargain.ResolveBubble(bargain.CorrectTalkId, false);
    bargain.TrySignDeal(20);
    var mortgage = new MortgageSim();
    mortgage.AddLoan(1, 40, 12, 100);
    var guard = 0;
    while (mortgage.TotalLoansCoins > 0 && guard++ < 500) mortgage.SettleMonth(78, 100, 0f);
    return new FullDrive
    {
        Ledger = ledger,
        Bargain = bargain,
        Mortgage = mortgage,
        Iaa = new IaaSim(),
    };
}

static string WallLinesJoin(FullDrive d)

```

```

    {
        var sb = new StringBuilder();
        var entries = AchieveSim.Derive(d.Ledger, d.Bargain, d.Mortgage, d.Iaa, 0f);
        for (var i = 0; i < entries.Count; i++)
        {
            if (i > 0) sb.Append('#');
            sb.Append(AchieveSim.ComposeWallLine(entries[i]));
        }
        sb.Append('#').Append(AchieveSim.ComposeWallCount(
            AchieveSim.UnlockedCount(d.Ledger, d.Bargain, d.Mortgage, d.Iaa, 0f),
            AchieveSim.Total));
        return sb.ToString();
    }

    static void WriteReport()
    {
        var sb = new StringBuilder();
        sb.AppendLine("ALL_GREEN passed=" + _passed + " failed=" + _failed);
        foreach (var l in Lines) sb.AppendLine(l);
        var dir = "Logs";
        if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
        File.WriteAllText(Path.Combine(dir, "g16_achieve_check_report.txt"), sb.ToString());
        if (_failed > 0) throw new Exception("AchieveCheck failed=" + _failed);
        UnityEngine.Debug.Log("AchieveCheck ALL_GREEN passed=" + _passed + " failed=" + _failed);
    }
}

// ===== FILE: Assets\\Game\\Scripts\\Editor\\AlbumCheck.cs =====
// G16 AlbumCheck - T009 codex album + share gate (slice A: data layer
// AL1-AL11 + slice B: view law AL12-AL14; T011: ten-card event deck laws
// AL15-AL17). Pure C#, zero-scene: four derived
// albums (property / tenant / event / review-literature, SYS09
// AlbumEntry{stem,unlocked}), the share dual channel with the "Tycoon
// Annual Report" long-image payload (X032 offline shape 2 - local
// composition, zero server), evt_share_card funnel integrity (one event
// per successful share with the card_id field, refuse = zero emissions),
// the codex desk view faces (tab / body / status / share-pair visibility
// + the view TryShare funnel walk), double-run determinism, copy-face
// coverage + U129 policy-word scan on the new ZH face.
// Report: Logs/g16_album_check_report.txt (first line
// ALL_GREEN passed=N failed=M).
using System;
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Text;
using System.Text.RegularExpressions;
using Sy.Telemetry;
using UnityEditor;

```

```

namespace Sy.CrazyEstate
{
    public static class AlbumCheck
    {
        static readonly List<string> Lines = new List<string>();
        static int _passed;
        static int _failed;

        // U129 banned-word dictionary (script "Golden Twenty Years" section 3b),
        // written as unicode escapes so the source stays pure ASCII (N8 rule).
        const string PolicyWordPattern =
            "\u653F\u7B56|\u65B0\u653F|\u8C03\u63A7|\u68DA\u6539|\u65E7\u6539|\u9650\u8D2D|\u9650\u8D37|\u9650\u552E"
            + "|\u623F\u4EA7\u7A0E|\u6301\u6709\u7A0E|\u5951\u7A0E|\u79DF\u8D2D\u5E76\u4E3E|\u4FDD\u4EA4\u697C"
            +
            "|\u623F\u4F4F\u4E0D\u7092|\u4E09\u6761\u7EA2\u7EBF|\u516C\u79EF\u91D1|\u843D\u6237|\u5F81\u6536|\u7ECF\u6D4E\u9002\u7528";

        static readonly string[] NewZhKeys =
        {
            "album_open", "album_title", "album_cat_property", "album_cat_tenant",
            "album_cat_event", "album_cat_review", "album_locked", "album_empty",
            "share_open", "share_card", "share_report_title", "share_done",
            "report_line_owned", "report_line_value", "report_line_rent",
            "report_line_tenants", "report_line_events", "report_line_market",
        };

        static void Assert(string id, bool ok, string detail)
        {
            if (ok) _passed++; else _failed++;
            Lines.Add("[ " + (ok ? "PASS" : "FAIL") + " ] " + id + " - " + detail);
        }

        // Funnel probe: captures every Track call for field-level audit.
        sealed class ProbeSink : TelemetryService.ISink
        {
            public readonly List<KeyValuePair<string, Dictionary<string, string>>> Events =
                new List<KeyValuePair<string, Dictionary<string, string>>>();

            public void Write(string evtId, Dictionary<string, string> fields)
                => Events.Add(new KeyValuePair<string, Dictionary<string, string>>(
                    evtId, new Dictionary<string, string>(fields)));
        }

        public static void Run()
        {
            try
            {
                // AL1 anchor law (SYS09 + requirements doc section 8): four
                // albums with live capacities (3 listings / 10 tenants /
                // 10 event cards / 10 review slots), unique non-empty stems,
                // funnel id evt_share_card on the telemetry package enum,
                // dual-channel card ids.
            }
        }
    }
}

```

```

var empty = new RentLedger();
var propFace = AlbumSim.PropertyAlbum(empty);
var tenFace = AlbumSim.TenantAlbum(empty);
var evtFace = AlbumSim.EventAlbum(null);
var revFace = AlbumSim.ReviewAlbum(empty);
var stemsUnique = UniqueStems(propFace) && UniqueStems(tenFace)
    && UniqueStems(evtFace) && UniqueStems(revFace);
Assert("AL1_anchor_law",
    propFace.Count == ListingTable.Listings.Length
    && tenFace.Count == TenantTable.Count
    && evtFace.Count == EventCardTable.Count
    && revFace.Count == TenantTable.Count
    && EventIds.ShareCard == "evt_share_card"
    && AlbumSim.CardIdShare == "share_card"
    && AlbumSim.CardIdReport == "annual_report"
    && stemsUnique,
    "albums=" + propFace.Count + "/" + tenFace.Count + "/"
    + evtFace.Count + "/" + revFace.Count
    + " funnel=" + EventIds.ShareCard + " stemsUnique=" + stemsUnique);

// AL2 property album law: empty portfolio keeps every slot
// locked; owning listings 1 and 3 unlocks exactly those two
// stems and the house copy keys resolve.
var own = new RentLedger();
own.AddOwned(1);
own.AddOwned(3);
var emptyPropUnlocked = CountUnlocked(AlbumSim.PropertyAlbum(new RentLedger()));
var ownPropStems = UnlockedStamp(AlbumSim.PropertyAlbum(own));
var houseKeysOk = CopyTable.TryGetRaw("house_h01", out var h1)
    && !string.IsNullOrEmpty(h1)
    && CopyTable.TryGetRaw("house_h03", out var h3)
    && !string.IsNullOrEmpty(h3);
Assert("AL2_property_album_law",
    emptyPropUnlocked == 0 && ownPropStems == "prop_01|prop_03"
    && houseKeysOk,
    "emptyUnlocked=" + emptyPropUnlocked + " stems=" + ownPropStems
    + " houseKeys=" + houseKeysOk);

// AL3 tenant album law: unlock mirrors the deterministic
// roster assignment (row0 -> seat 8, row1 -> seat 3).
var ownTenStems = UnlockedStamp(AlbumSim.TenantAlbum(own));
Assert("AL3_tenant_album_law",
    ownTenStems == "tenant_03|tenant_08",
    "stems=" + ownTenStems + " (deterministic seats 8/3)");

// AL4 review-literature mirror law: review slots unlock
// exactly with their tenant slot, and every unlocked review
// key resolves on both locale faces.
var ownTen = AlbumSim.TenantAlbum(own);
var ownRev = AlbumSim.ReviewAlbum(own);

```

```

var mirrorOk = true;
var reviewKeysOk = true;
var zh = CopyTableZh.Snapshot();
for (var i = 0; i < ownRev.Count; i++)
{
    if (ownRev[i].Unlocked != ownTen[i].Unlocked) mirrorOk = false;
    if (ownRev[i].Unlocked)
    {
        var t = TenantTable.GetTenant(i + 1);
        if (!CopyTable.TryGetRaw(t.ReviewKey, out var rEn) || string.IsNullOrEmpty(rEn))
            reviewKeysOk = false;
        if (!zh.TryGetValue(t.ReviewKey, out var rZh) || string.IsNullOrEmpty(rZh))
            reviewKeysOk = false;
    }
}
var rosterProblems = TenantTable.Validate(new List<string>());
Assert("AL4_review_mirror_law",
    mirrorOk && reviewKeysOk && rosterProblems.Count == 0,
    "mirror=" + mirrorOk + " reviewKeys=" + reviewKeysOk
    + " rosterProblems=" + rosterProblems.Count);

// AL5 event album law: a fresh stream locks all three slots;
// the guaranteed first card (E01 at 8s) unlocks exactly evt_e01
// and the album mirrors IsCollected card by card.
var fresh = new EventStream();
var freshUnlocked = CountUnlocked(AlbumSim.EventAlbum(fresh));
fresh.Tick(9f, 0);
var afterStems = UnlockedStamp(AlbumSim.EventAlbum(fresh));
var mirrorCards = true;
var afterFace = AlbumSim.EventAlbum(fresh);
for (var i = 0; i < EventCardTable.Cards.Length; i++)
    if (afterFace[i].Unlocked != fresh.IsCollected(EventCardTable.Cards[i].Id))
        mirrorCards = false;
Assert("AL5_event_album_law",
    freshUnlocked == 0 && afterStems == "evt_e01" && mirrorCards,
    "freshUnlocked=" + freshUnlocked + " after=" + afterStems
    + " mirror=" + mirrorCards);

// AL6 annual report law (channel 2): the long-image payload
// is the title plus six exact live stat lines - 2 homes,
// 160 wan portfolio value (40 + 120), 408 coins lifetime rent
// (2 beats x (60 + 144)), 2 tenants, 1/N event codex (N = the
// live event deck size), and the live market phase name.
var ledger = new RentLedger();
ledger.AddOwned(1);
ledger.AddOwned(3);
ledger.AccrueBeats(2);
var stream = new EventStream();
stream.Tick(9f, 0);
var cycle = new MarketCycle(20261003);

```

```

var report = AlbumSim.ComposeAnnualReport(ledger, stream, cycle);
var reportLines = report.Split('\n');
var inv = CultureInfo.InvariantCulture;
var l0 = CopyTable.Get("share_report_title");
var l1 = string.Format(inv, CopyTable.Get("report_line_owned"), 2);
var l2 = string.Format(inv, CopyTable.Get("report_line_value"), 160);
var l3 = string.Format(inv, CopyTable.Get("report_line_rent"), 408);
var l4 = string.Format(inv, CopyTable.Get("report_line_tenants"), 2);
var l5 = string.Format(inv, CopyTable.Get("report_line_events"), 1, EventCardTable.Count);
var l6 = string.Format(inv, CopyTable.Get("report_line_market"),
    CopyTable.Get(cycle.PhaseNameKey));
Assert("AL6_annual_report_law",
    reportLines.Length == 7
    && reportLines[0] == l0 && reportLines[1] == l1 && reportLines[2] == l2
    && reportLines[3] == l3 && reportLines[4] == l4 && reportLines[5] == l5
    && reportLines[6] == l6,
    "lines=" + reportLines.Length + " owned=2 value=160 rent=408"
    + " tenants=2 events=1/" + EventCardTable.Count);

// AL7 share card law (channel 1): an empty portfolio composes
// nothing (null, zero-tears refuse); a live portfolio fills
// both placeholders in order with no brace residue.
var nullCard = AlbumSim.ComposeShareCard(new RentLedger());
var card = AlbumSim.ComposeShareCard(ledger);
var expectedCard = string.Format(inv, CopyTable.Get("share_card"), 2, 408);
Assert("AL7_share_card_law",
    nullCard == null && card == expectedCard && card.IndexOf('{') < 0,
    "nullCard=" + (nullCard == null) + " match=" + (card == expectedCard));

// AL8 share funnel law (requirements doc section 8): every
// successful share walks exactly one evt_share_card event
// with the card_id; the empty-portfolio refuse emits nothing.
var tel = new TelemetryService();
var probe = new ProbeSink();
tel.AddSink(probe);
var refusedEmpty = AlbumSim.Share(new RentLedger(), tel, AlbumSim.CardIdShare);
var okShare = AlbumSim.Share(ledger, tel, AlbumSim.CardIdShare);
var okReport = AlbumSim.Share(ledger, tel, AlbumSim.CardIdReport);
Assert("AL8_share_funnel_law",
    !refusedEmpty && okShare && okReport && probe.Events.Count == 2
    && probe.Events[0].Key == "evt_share_card"
    && probe.Events[0].Value["card_id"] == "share_card"
    && probe.Events[1].Key == "evt_share_card"
    && probe.Events[1].Value["card_id"] == "annual_report",
    "refused=" + refusedEmpty + " shares=" + okShare + "/" + okReport
    + " events=" + probe.Events.Count);

// IA9-style determinism law: two sims driven by the same call
// script land byte-identical report + card + funnel + album
// stamps.

```

```

var probeA = new ProbeSink();
var telA = new TelemetryService();
telA.AddSink(probeA);
var probeB = new ProbeSink();
var telB = new TelemetryService();
telB.AddSink(probeB);
var fpA = DriveShareScript(telA, probeA);
var fpB = DriveShareScript(telB, probeB);
Assert("AL9_determinism_law",
    fpA == fpB && probeA.Events.Count == probeB.Events.Count,
    "fpMatch=" + (fpA == fpB) + " events=" + probeA.Events.Count
    + "/" + probeB.Events.Count);

// AL10 copy-face law: the 18 new album/share keys exist on
// both locale faces, sit on the single-authority key list,
// and the shared validator (coverage + non-empty + placeholder
// parity) stays clean.
var keyListOk = true;
var faceOk = true;
foreach (var k in NewZhKeys)
{
    if (Array.IndexOf(CopyTable.ZhKeyList, k) < 0) keyListOk = false;
    if (!CopyTable.TryGetRaw(k, out var en) || string.IsNullOrEmpty(en)) faceOk = false;
    if (!zh.TryGetValue(k, out var z) || string.IsNullOrEmpty(z)) faceOk = false;
}
var problems = CopyTable.Validate(new List<string>(), zh);
Assert("AL10_copy_face_law",
    keyListOk && faceOk && problems.Count == 0,
    "keyList=" + keyListOk + " faces=" + faceOk
    + " validatorProblems=" + problems.Count + " keys=" + NewZhKeys.Length);

// AL11 policy-word scan (U129): the new ZH comedy copy stays
// clean of the banned dictionary; the positive control proves
// the scanner is alive.
var bannedHits = 0;
foreach (var k in NewZhKeys)
    if (zh.TryGetValue(k, out var v) && Regex.IsMatch(v, PolicyWordPattern))
        bannedHits++;
var control = Regex.IsMatch("\u8C03\u63A7", PolicyWordPattern);
Assert("AL11_policy_scan_law",
    bannedHits == 0 && control,
    "hits=" + bannedHits + " control=" + control);

// T009 slice-B view law: the codex desk composes from the
// same single truths frozen above (no second truth exists).

// AL12 tab-face law: category tabs read the cat copy plus
// the live unlock count, invariant digits.
var tabProp = AlbumViewHost.ComposeTabFace("album_cat_property", 1, ListingTable.Listings.Length);
var tabPropExpected = CopyTable.Get("album_cat_property")

```

```

    + " 1/" + ListingTable.Listings.Length.ToString(inv);
var tabEvt = AlbumViewHost.ComposeTabFace("album_cat_event", 0, 3);
var tabEvtExpected = CopyTable.Get("album_cat_event") + " 0/3";
Assert("AL12_view_tab_face_law",
    tabProp == tabPropExpected && tabEvt == tabEvtExpected,
    "propTab=" + (tabProp == tabPropExpected) + " evtTab=" + (tabEvt == tabEvtExpected));

// AL13 view body law: unlocked lines read the resolved
// names, locked lines read the locked copy, and the blank
// codex status face composes only until something unlocks.
var lockedCopy = CopyTable.Get("album_locked");
var propBody = AlbumViewHost.ComposeAlbumBody(AlbumSim.PropertyAlbum(own),
    delegate(int i) { return CopyTable.Get(ListingTable.Listings[i].HouseKey); });
// Expected body derives from the full table (X850 H04-H10 expansion
// migration): rows 1 and 3 unlocked -> resolved names, rest locked.
var bodySb = new StringBuilder();
for (var i = 0; i < ListingTable.Listings.Length; i++)
{
    if (i > 0) bodySb.Append('\n');
    bodySb.Append(i == 0 || i == 2
        ? CopyTable.Get(ListingTable.Listings[i].HouseKey)
        : lockedCopy);
}
var propBodyExpected = bodySb.ToString();
var entryLocked = AlbumViewHost.ComposeEntryLine(
    new AlbumEntry { Stem = "x", Unlocked = false }, "name");
var entryOpen = AlbumViewHost.ComposeEntryLine(
    new AlbumEntry { Stem = "x", Unlocked = true }, "name");
var statusBlank = AlbumViewHost.ComposeStatusFace(false);
var statusLive = AlbumViewHost.ComposeStatusFace(true);
Assert("AL13_view_body_law",
    propBody == propBodyExpected && entryLocked == lockedCopy && entryOpen == "name"
    && statusBlank == CopyTable.Get("album_empty") && statusLive == string.Empty,
    "body=" + (propBody == propBodyExpected)
    + " statusBlank=" + (statusBlank == CopyTable.Get("album_empty")));

// AL14 view share law: the share pair stays hidden on an
// empty portfolio; both channels walk the same funnel and
// hand the view the exact composed payload.
var viewTel = new TelemetryService();
var viewProbe = new ProbeSink();
viewTel.AddSink(viewProbe);
string cardPayload;
string reportPayload;
string emptyPayload;
var emptyShare = AlbumViewHost.TryShare(AlbumSim.CardIdShare,
    new RentLedger(), stream, cycle, viewTel, out emptyPayload);
var cardShare = AlbumViewHost.TryShare(AlbumSim.CardIdShare,
    ledger, stream, cycle, viewTel, out cardPayload);
var reportShare = AlbumViewHost.TryShare(AlbumSim.CardIdReport,

```

```

    ledger, stream, cycle, viewTel, out reportPayload);
Assert("AL14_view_share_law",
    !emptyShare && emptyPayload == null
    && !AlbumViewHost.ShareButtonsVisible(new RentLedger())
    && AlbumViewHost.ShareButtonsVisible(ledger)
    && cardShare && reportShare
    && cardPayload == AlbumSim.ComposeShareCard(ledger)
    && reportPayload == AlbumSim.ComposeAnnualReport(ledger, stream, cycle)
    && viewProbe.Events.Count == 2
    && viewProbe.Events[0].Value["card_id"] == "share_card"
    && viewProbe.Events[1].Value["card_id"] == "annual_report",
    "emptyRefuse=" + !emptyShare + " pairVisible=" + AlbumViewHost.ShareButtonsVisible(ledger)
    + " shares=" + cardShare + "/" + reportShare + " events=" + viewProbe.Events.Count);

// T011 ten-card deck law: the event roster carries the full
// E01-E10 deck in id order with both locale faces resolvable
// (schema via the shared validator), and the event album
// mirrors it slot for slot with canonical evt_eNN stems.
var zhDeck = CopyTableZh.Snapshot();
var deckProblems = new List<string>();
EventCardTable.Validate(deckProblems, zhDeck);
var deckFace = AlbumSim.EventAlbum(null);
var deckStemsOk = deckFace.Count == 10;
for (var i = 0; i < deckFace.Count; i++)
    if (deckFace[i].Stem != "evt_e" + (i + 1).ToString("D2", CultureInfo.InvariantCulture))
        deckStemsOk = false;
Assert("AL15_event_deck_ten_law",
    EventCardTable.Count == 10 && deckProblems.Count == 0 && deckStemsOk,
    "deck=" + EventCardTable.Count + " schema=" + deckProblems.Count
    + " stems=" + deckStemsOk);

// T011 full-collection law: a long owned-home drive sees
// every one of the ten cards (rare jackpot included, first
// hit epoch 7) and both popup faces resolve non-empty on
// each locale face for every collected card.
var full = new EventStream();
var guard16 = 0;
while (full.CollecteCount < EventCardTable.Count && guard16++ < 4000)
    full.Tick(300f, 1);
var popupFacesOk = true;
for (var i = 0; i < EventCardTable.Cards.Length; i++)
{
    var def = EventCardTable.Cards[i];
    if (!full.IsCollected(def.Id)) { popupFacesOk = false; continue; }
    var tEn = EventPopupHost.ResolvePopupTitle(def.Id);
    var bEn = EventPopupHost.ResolvePopupBody(def.Id);
    if (string.IsNullOrEmpty(tEn) || string.IsNullOrEmpty(bEn)) popupFacesOk = false;
    if (!zhDeck.TryGetValue(def.TitleKey, out var tZh) || string.IsNullOrEmpty(tZh))
        popupFacesOk = false;
    if (!zhDeck.TryGetValue(def.BodyKey, out var bZh) || string.IsNullOrEmpty(bZh))

```

```

        popupFacesOk = false;
    }
    Assert("AL16_full_collection_law",
        full.CollecteCount == EventCardTable.Count && popupFacesOk && guard16 <= 4000,
        "collected=" + full.CollecteCount + "/" + EventCardTable.Count
        + " epochs=" + full.Epoch + " popupFaces=" + popupFacesOk);

    // T011 common-pool spread law: with no owned home the jackpot
    // stays gated off, yet the rotation alone still covers all
    // nine common cards inside 300 windows - the codex never
    // stalls on a single card (verified spread, zero RNG).
    var spread = new EventStream();
    for (var e = 0; e < 300; e++) spread.Tick(300f, 0);
    Assert("AL17_common_pool_spread_law",
        spread.CollecteCount == 9
        && !spread.IsCollected(EventCardId.DemolitionJackpot),
        "collected=" + spread.CollecteCount + "/9 epochs=" + spread.Epoch);
}
catch (Exception e)
{
    _failed++;
    Lines.Add("[FAIL] AL0_exception - " + e.Message);
}

WriteReport();
}

static string DriveShareScript(TelemetryService tel, ProbeSink probe)
{
    var l = new RentLedger();
    l.AddOwned(1);
    l.AddOwned(3);
    l.AccrueBeats(2);
    var s = new EventStream();
    s.Tick(9f, 0);
    var c = new MarketCycle(20261003);
    var sb = new StringBuilder();
    sb.Append(AlbumSim.ComposeAnnualReport(l, s, c));
    sb.Append('#').Append(AlbumSim.ComposeShareCard(1));
    sb.Append('#').Append(AlbumSim.Share(l, tel, AlbumSim.CardIdShare));
    sb.Append('#').Append(AlbumSim.Share(l, tel, AlbumSim.CardIdReport));
    sb.Append('#').Append(probe.Events.Count);
    sb.Append('#').Append(UnlockedStamp(AlbumSim.PropertyAlbum(1)));
    sb.Append('#').Append(UnlockedStamp(AlbumSim.TenantAlbum(1)));
    sb.Append('#').Append(UnlockedStamp(AlbumSim.EventAlbum(s)));
    sb.Append('#').Append(UnlockedStamp(AlbumSim.ReviewAlbum(1)));
    return sb.ToString();
}

static int CountUnlocked(List<AlbumEntry> entries)

```

```

    {
        var n = 0;
        for (var i = 0; i < entries.Count; i++)
            if (entries[i].Unlocked) n++;
        return n;
    }

    static string UnlockedStamp(List<AlbumEntry> entries)
    {
        var sb = new StringBuilder();
        for (var i = 0; i < entries.Count; i++)
        {
            if (!entries[i].Unlocked) continue;
            if (sb.Length > 0) sb.Append('|');
            sb.Append(entries[i].Stem);
        }
        return sb.ToString();
    }

    static bool UniqueStems(List<AlbumEntry> entries)
    {
        var seen = new HashSet<string>();
        for (var i = 0; i < entries.Count; i++)
        {
            if (string.IsNullOrEmpty(entries[i].Stem)) return false;
            if (!seen.Add(entries[i].Stem)) return false;
        }
        return true;
    }

    static void WriteReport()
    {
        var sb = new StringBuilder();
        sb.AppendLine("ALL_GREEN passed=" + _passed + " failed=" + _failed);
        foreach (var l in Lines) sb.AppendLine(l);
        var dir = "Logs";
        if (!Directory.Exists(dir)) Directory.CreateDirectory(dir);
        File.WriteAllText(Path.Combine(dir, "g16_album_check_report.txt"), sb.ToString());
        if (_failed > 0) throw new Exception("AlbumCheck failed=" + _failed);
        UnityEngine.Debug.Log("AlbumCheck ALL_GREEN passed=" + _passed + " failed=" + _failed);
    }
}

// ===== FILE: Assets\\Game\\Scripts\\Editor\\ArtWireCheck.cs =====
// G16 ArtWireCheck (X1209 art wiring face gate): the harvested AA-016 pixel
// kit (X1203 wave2, 15/15 ArtGateCheck PASS) must be CONSUMED by the S16_Main
// view, not just landed in Art/ (X1207 U088 first-run root cause: the board
// frame was a UI-text shell with zero pixel city visible = art wiring gap,
// a real P0 that blocked the T016 DoD close). Faces:
// W1 catalog face - the three wired stems resolve non-null via ArtRuntime

```

```

//      (ArtSetup-built catalog, Resources lookup);
//      W2 sprite-border face - the stretch-consumed sprites carry meta borders
//      (panel 64 / button 32) so Sliced keeps corner art under stretch;
//      W3 host wiring face - the real ListingViewHost BuildUi consumes the
//      sprites (city backdrop + board panel + primary buttons) with the
//      X027 null-fallback contract intact;
//      W4 paint face - a feed-style real-host rig render (X1192 pattern)
//      proves the wired faces actually paint on canvas: dense probes on
//      the city band, the board panel and the primary button.
// Report: Logs/g16_artwire_check_report.txt (first line ALL_GREEN).
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using Sy.Core;
using UnityEditor;
using UnityEngine;
using UnityEngine.UI;

namespace Sy.CrazyEstate
{
    public static class ArtWireCheck
    {
        {
            const int FrameW = 720, FrameH = 1280; // ListingViewHost reference frame
            const int Uilayer = 5; // "UI": rig camera is the only observer
            static readonly Color RigBg = new Color(0.02f, 0.05f, 0.09f, 1f);
            const float ProbeTol = 0.10f; // per-channel distance from rig bg

            static readonly List<string> Lines = new List<string>();
            static int _passed;
            static int _failed;

            static void Assert(string id, bool ok, string detail)
            {
                if (ok) _passed++; else _failed++;
                Lines.Add("[ " + (ok ? "PASS" : "FAIL") + " ] " + id + " - " + detail);
            }

            public static void Run()
            {
                try
                {
                    // ---- W1 catalog face: the wired stems must resolve ----
                    var cityArt = ArtRuntime.Get("scene_city_skyline");
                    var panelArt = ArtRuntime.Get("ui_panel_main");
                    var btnArt = ArtRuntime.Get("ui_btn_primary");
                    Assert("W1_catalog_stems", cityArt != null && panelArt != null && btnArt != null,
                        "city=" + (cityArt != null) + " panel=" + (panelArt != null)
                        + " btn=" + (btnArt != null) + " (ArtRuntime catalog lookup, art wiring face)");
                }
            }
        }
    }
}

```

```

// ---- W2 sprite-border face: Sliced needs meta borders ----
var panelBorderOk = panelArt != null
  && panelArt.border.x >= 48f && panelArt.border.y >= 48f
  && panelArt.border.z >= 48f && panelArt.border.w >= 48f;
Assert("W2_panel_border", panelBorderOk,
  "panel border=" + (panelArt != null ? panelArt.border.ToString("F0") : "n/a")
  + " (64px anchor keeps the mushroom corner clusters under stretch)");
var btnBorderOk = btnArt != null
  && btnArt.border.x >= 16f && btnArt.border.y >= 16f
  && btnArt.border.z >= 16f && btnArt.border.w >= 16f;
Assert("W3_button_border", btnBorderOk,
  "button border=" + (btnArt != null ? btnArt.border.ToString("F0") : "n/a")
  + " (32px anchor keeps the capsule caps under stretch)");

// ---- W3 host wiring face: real BuildUi consumes the art ----
// Edit mode keeps Start() inert, so BuildUi is driven through
// reflection like the QG-ext feed frame (X1192 pattern).
var holder = new GameObject("ArtWireHostSeat");
Image backdropFace = null, panelFace = null, openBtnFace = null;
GameObject canvasGo = null;
try
{
  var host = holder.AddComponent<ListingViewHost>();
  var hostType = typeof(ListingViewHost);
  const System.Reflection.BindingFlags Np =
    System.Reflection.BindingFlags.NonPublic | System.Reflection.BindingFlags.Instance;
  hostType.GetMethod("BuildUi", Np).Invoke(host, null);
  canvasGo = GameObject.Find("ListingCanvas");
  Assert("W4_host_canvas_built", canvasGo != null,
    "ListingCanvas found=" + (canvasGo != null) + " (real board host, programmatic UGUI)");
  if (canvasGo != null)
  {
    var crt = (RectTransform)canvasGo.transform;
    var bd = crt.Find("CityBackdrop");
    var bp = crt.Find("BoardPanel");
    var ob = crt.Find("BoardOpenButton");
    backdropFace = bd != null ? bd.GetComponent<Image>() : null;
    panelFace = bp != null ? bp.GetComponent<Image>() : null;
    openBtnFace = ob != null ? ob.GetComponent<Image>() : null;
    Assert("W5_backdrop_wired", backdropFace != null && backdropFace.sprite == cityArt,
      "CityBackdrop sprite==cityArt " + (backdropFace != null && backdropFace.sprite == cityArt));
    Assert("W6_panel_wired", panelFace != null && panelFace.sprite == panelArt
      && panelFace.type == Image.Type.Sliced,
      "BoardPanel sprite==panelArt " + (panelFace != null && panelFace.sprite == panelArt)
      + " type=" + (panelFace != null ? panelFace.type.ToString() : "n/a"));
    Assert("W7_button_wired", openBtnFace != null && openBtnFace.sprite == btnArt,
      "BoardOpenButton sprite==btnArt " + (openBtnFace != null && openBtnFace.sprite == btnArt));

    // X1234 T016-d7 board HUD chrome law (U088 wave-5 A2
    // gap: no persistent currency HUD, banners sparse):

```

```

// the cash banner must ride the real host - panel
// chrome + coin icon + live wallet face + anchor
// accent, boot-hidden and open-shown with the desk
// (SetBoardOpen drives it, reflection-driven here).
var cashBarFace = crt.Find("BoardCashBar");
var cashFaceT = cashBarFace != null ? cashBarFace.Find("CashFace") : null;
var cashIconT = cashBarFace != null ? cashBarFace.Find("CashCoinIcon") : null;
var cashAccentFace = crt.Find("BoardCashAccent");
var cashBarImg = cashBarFace != null ? cashBarFace.GetComponent<Image>() : null;
var cashFaceTxt = cashFaceT != null ? cashFaceT.GetComponent<Text>() : null;
var cashIconImg = cashIconT != null ? cashIconT.GetComponent<Image>() : null;
var cashAccentImg = cashAccentFace != null ? cashAccentFace.GetComponent<Image>() : null;
var coinArt = ArtRuntime.Get("icon_coin");
bool cashBootHidden = cashBarFace != null && !cashBarFace.gameObject.activeSelf
    && cashAccentFace != null && !cashAccentFace.gameObject.activeSelf;
hostType.GetMethod("SetBoardOpen", Np).Invoke(host, new object[] { true });
bool cashOpenShown = cashBarFace != null && cashBarFace.gameObject.activeSelf
    && cashAccentFace != null && cashAccentFace.gameObject.activeSelf;
bool cashChromeWired = cashBarImg != null && panelArt != null
    && cashBarImg.sprite == panelArt && cashBarImg.type == Image.Type.Sliced
    && !cashBarImg.raycastTarget
    && cashIconImg != null && coinArt != null && cashIconImg.sprite == coinArt
    && !cashIconImg.raycastTarget;
bool cashFaceTrue = cashFaceTxt != null && !string.IsNullOrEmpty(cashFaceTxt.text)
    && cashFaceTxt.text == ListingViewHost.ComposeCashFace(RentLedger.StartCashWan, 0);
Assert("W26_board_cash_bar_wired",
    cashBootHidden && cashOpenShown && cashChromeWired && cashFaceTrue
    && cashAccentImg != null && cashAccentImg.color == G16StyleAnchor.AccentColor,
    "boot-hidden=" + cashBootHidden + " open-shown=" + cashOpenShown
    + " panel-sliced=" + (cashBarImg != null && cashBarImg.sprite == panelArt
        && cashBarImg.type == Image.Type.Sliced)
    + " coin=" + (cashIconImg != null && cashIconImg.sprite == coinArt)
    + " face=\"" + (cashFaceTxt != null ? cashFaceTxt.text : "n/a")
    + "\" accent-anchor=" + (cashAccentImg != null
        && cashAccentImg.color == G16StyleAnchor.AccentColor)
    + " (persistent wallet HUD on the board desk)");
}
}
finally
{
    // keep the host for the paint face; both die in the outer finally
}

// ---- X1215 duel wiring face: the A3 bargain flow frame must
// consume the room art too (X1211 judge advisory: art-thin
// flow screens keep the U088 verdict red-flip prone). Same
// reflection pattern as the board face above.
var roomArt = ArtRuntime.Get("scene_bargain_room");
Assert("W11_room_catalog", roomArt != null,
    "room=" + (roomArt != null) + " (bargain-room scene stem resolves)");

```

```

GameObject duelCanvasGo = null;
var duelSeat = new GameObject("BargainWireHostSeat");
try
{
    var duelHost = duelSeat.AddComponent<BargainViewHost>();
    var duelType = typeof(BargainViewHost);
    const System.Reflection.BindingFlags DuelNp =
        System.Reflection.BindingFlags.NonPublic | System.Reflection.BindingFlags.Instance;
    duelType.GetMethod("BuildUi", DuelNp).Invoke(duelHost, null);
    duelCanvasGo = GameObject.Find("BargainCanvas");
    Assert("W12_duel_canvas_built", duelCanvasGo != null,
        "BargainCanvas found=" + (duelCanvasGo != null) + " (real duel host, programmatic UGUI)");
    if (duelCanvasGo != null)
    {
        var drt = (RectTransform)duelCanvasGo.transform;
        var roomFace = drt.Find("BargainBackdrop");
        var scrimFace = drt.Find("BargainScrim");
        var roomImg = roomFace != null ? roomFace.GetComponent<Image>() : null;
        var scrimImg = scrimFace != null ? scrimFace.GetComponent<Image>() : null;
        Assert("W13_duel_backdrop_wired",
            roomImg != null && roomImg.sprite == roomArt && roomImg.preserveAspect
            && !roomImg.raycastTarget && roomFace.GetSiblingIndex() == 0,
            "room sprite==roomArt " + (roomImg != null && roomImg.sprite == roomArt)
            + " aspect=" + (roomImg != null && roomImg.preserveAspect)
            + " raycast=" + (roomImg != null && roomImg.raycastTarget)
            + " sibling=" + (roomFace != null ? roomFace.GetSiblingIndex().ToString() : "n/a"));
        Assert("W14_duel_scrim_face",
            scrimImg != null && scrimImg.sprite == null
            && scrimImg.color.a >= 0.30f && scrimImg.color.a <= 0.60f
            && !scrimImg.raycastTarget,
            "scrim alpha=" + (scrimImg != null ? scrimImg.color.a.ToString("F2") : "n/a")
            + " (dark veil under the toast/store text, above the room art)");

        // X1218 slice-2 flow dressing laws (T016-d): the A1
        // hook city band + the A4 settle card face ride the
        // same duel canvas - boot-phase state is asserted so
        // the hook->room swap contract stays machine-frozen.
        var hookCityFace = drt.Find("HookCityBackdrop");
        var hookCityImg = hookCityFace != null ? hookCityFace.GetComponent<Image>() : null;
        Assert("W15_hook_city_wired",
            hookCityImg != null && hookCityImg.sprite == cityArt
            && hookCityImg.preserveAspect && !hookCityImg.raycastTarget
            && hookCityFace.gameObject.activeSelf,
            "hook city sprite==cityArt " + (hookCityImg != null && hookCityImg.sprite == cityArt)
            + " aspect=" + (hookCityImg != null && hookCityImg.preserveAspect)
            + " active=" + (hookCityFace != null ? hookCityFace.gameObject.activeSelf.ToString() : "n/a")
            + " (boot hook face on the city skyline band)");

        // X1234 T016-d7 A1 readability law (U088 wave-5
        // advisory: the title sat low-contrast on the sky

```

```

// band): the hook face carries a hard drop shadow -
// the cheapest contrast fix, frozen here so the frame
// never ships a washed-out welcome.
var hookTitleFace = drt.Find("HookFace");
var hookTitleShadow = hookTitleFace != null
    ? hookTitleFace.GetComponent<Shadow>() : null;
Assert("W27_hook_title_shadow_law",
    hookTitleShadow != null
    && hookTitleShadow.effectColor.a >= 0.5f
    && hookTitleShadow.effectColor.r <= 0.2f
    && hookTitleShadow.effectDistance.sqrMagnitude > 0f,
    "hook shadow=" + (hookTitleShadow != null)
    + " alpha=" + (hookTitleShadow != null
        ? hookTitleShadow.effectColor.a.ToString("F2") : "n/a")
    + " dist=" + (hookTitleShadow != null
        ? hookTitleShadow.effectDistance.ToString() : "n/a")
    + " (welcome copy readable over the lit city band)");
var settleCardFace = drt.Find("SettleCard");
var settleCardImg = settleCardFace != null ? settleCardFace.GetComponent<Image>() : null;
Assert("W16_settle_card_face",
    settleCardImg != null && settleCardImg.sprite == null
    && !settleCardImg.raycastTarget
    && !settleCardFace.gameObject.activeSelf,
    "settle card boot-inactive=" + (settleCardFace != null && !settleCardFace.gameObject.activeSelf)
    + " sprite-null=" + (settleCardImg != null && settleCardImg.sprite == null)
    + " (deal-close house card, hidden until the stamp lands)");
var luxuryCard = ArtRuntime.Get("house_luxury_tower");
var shabbyCard = ArtRuntime.Get("house_old_shabby");
Assert("W17_settle_stem_law",
    luxuryCard != null && shabbyCard != null
    && BargainViewHost.SettleCardStemFor(0) == "house_luxury_tower"
    && BargainViewHost.SettleCardStemFor(1) == "house_old_shabby"
    && BargainViewHost.SettleCardStemFor(5) == "house_luxury_tower",
    "stems resolve luxury=" + (luxuryCard != null) + " shabby=" + (shabbyCard != null)
    + " map 0->" + BargainViewHost.SettleCardStemFor(0)
    + " 1->" + BargainViewHost.SettleCardStemFor(1)
    + " 5->" + BargainViewHost.SettleCardStemFor(5)
    + " (guided=dream tower, H01 tier0 starter, H05 tier2 premium)");

// X1223 T016-d3 portrait-seat laws: the duel face
// wears its busts - the landlord flank (always on in
// the duel segment) + the gated agent flank (only
// CodexRoster agent_01 Tiezui Wang has a portrait;
// agents 02..05 honestly hide, X027 law).
var landlordArt = ArtRuntime.Get("unit_landlord_player");
var agentBustArt = ArtRuntime.Get("unit_agent_wang");
var playerSeatFace = drt.Find("PlayerPortraitSeat");
var agentSeatFace = drt.Find("AgentPortraitSeat");
var playerSeatImg = playerSeatFace != null ? playerSeatFace.GetComponent<Image>() : null;
var agentSeatImg = agentSeatFace != null ? agentSeatFace.GetComponent<Image>() : null;

```

```

Assert("W18_portrait_seats_wired",
    playerSeatImg != null && agentSeatImg != null
    && landlordArt != null && agentBustArt != null
    && playerSeatImg.sprite == landlordArt && playerSeatImg.preserveAspect
    && !playerSeatImg.raycastTarget
    && agentSeatImg.sprite == agentBustArt && agentSeatImg.preserveAspect
    && !agentSeatImg.raycastTarget
    && !playerSeatFace.gameObject.activeSelf && !agentSeatFace.gameObject.activeSelf,
    "stems landlord=" + (landlordArt != null) + " agent=" + (agentBustArt != null)
    + " aspect=" + (playerSeatImg != null && playerSeatImg.preserveAspect
    && agentSeatImg != null && agentSeatImg.preserveAspect)
    + " boot-hidden=" + (playerSeatFace != null && !playerSeatFace.gameObject.activeSelf
    && agentSeatFace != null && !agentSeatFace.gameObject.activeSelf)
    + " (hook segment hides both busts; sprites wired, aspect kept, raycast-transparent)");

// X1229 T016-d5 duel-chrome wiring laws (X1228 U088
// wave-4 Q3 density: the duel/settle mid bands were
// dark text shelves): the framed dialogue panel
// behind the bubble talk + the HUD accent strip under
// the price face. Boot-hidden with the hook segment.
var bubblePanelFace = drt.Find("BubbleDialoguePanel");
var duelAccentFace = drt.Find("DuelHudAccent");
var bubblePanelImg = bubblePanelFace != null ? bubblePanelFace.GetComponent<Image>() : null;
var duelAccentImg = duelAccentFace != null ? duelAccentFace.GetComponent<Image>() : null;
Assert("W20_duel_chrome_wired",
    bubblePanelImg != null && bubblePanelImg.sprite == panelArt
    && bubblePanelImg.type == Image.Type.Sliced && !bubblePanelImg.raycastTarget
    && !bubblePanelFace.gameObject.activeSelf
    && duelAccentImg != null && !duelAccentImg.raycastTarget
    && duelAccentImg.color == G16StyleAnchor.AccentColor
    && !duelAccentFace.gameObject.activeSelf,
    "bubble panel sprite==panelArt " + (bubblePanelImg != null && bubblePanelImg.sprite == panelArt)
    + " sliced=" + (bubblePanelImg != null && bubblePanelImg.type == Image.Type.Sliced)
    + " boot-inactive=" + (bubblePanelFace != null && !bubblePanelFace.gameObject.activeSelf)
    + " accent-anchor=" + (duelAccentImg != null && duelAccentImg.color == G16StyleAnchor.AccentColor)
    + " (framed dialogue + anchor accent strip, hook segment hides both)");

// X1229 T016-d5 settle-chrome wiring law: the framed
// panel behind the house card + the accent ribbon
// under it - boot-hidden until the stamp lands.
var settleFrameFace = drt.Find("SettleFramePanel");
var settleAccentFace = drt.Find("SettleAccentBar");
var settleFrameImg = settleFrameFace != null ? settleFrameFace.GetComponent<Image>() : null;
var settleAccentImg = settleAccentFace != null ? settleAccentFace.GetComponent<Image>() : null;
Assert("W22_settle_chrome_wired",
    settleFrameImg != null && settleFrameImg.sprite == panelArt
    && settleFrameImg.type == Image.Type.Sliced && !settleFrameImg.raycastTarget
    && !settleFrameFace.gameObject.activeSelf
    && settleAccentImg != null && !settleAccentImg.raycastTarget
    && settleAccentImg.color == G16StyleAnchor.AccentColor

```

```

    && !settleAccentFace.gameObject.activeSelf,
    "settle frame sprite==panelArt " + (settleFrameImg != null && settleFrameImg.sprite == panelArt)
+ " sliced=" + (settleFrameImg != null && settleFrameImg.type == Image.Type.Sliced)
+ " boot-inactive=" + (settleFrameFace != null && !settleFrameFace.gameObject.activeSelf)
+ " ribbon-anchor=" + (settleAccentImg != null && settleAccentImg.color == G16StyleAnchor.AccentColor)
+ " (framed certificate band + ribbon, hidden until the deal closes)");

// X1231 T016-d6 receipt-chip wiring law: four framed
// chips under the certificate band fill the settle
// mid-frame dead zone - labels ride stretch-to-parent
// child GOs (one-Graphic-per-GO law, X1192), boot
// hidden with the settle segment.
var chipRowWired = true;
var chipLabelsStretched = true;
var chipBootHidden = true;
for (var i = 0; i < 4; i++)
{
    var chipFace = drt.Find("HudChip" + i);
    if (chipFace == null) { chipRowWired = false; continue; }
    var chipImg = chipFace.GetComponent<Image>();
    var chipLabelRt = chipFace.Find("Label") as RectTransform;
    if (chipImg == null || chipImg.sprite != panelArt
        || chipImg.type != Image.Type.Sliced || chipImg.raycastTarget)
        chipRowWired = false;
    if (chipFace.gameObject.activeSelf) chipBootHidden = false;
    if (chipLabelRt == null
        || chipLabelRt.anchorMin != Vector2.zero
        || chipLabelRt.anchorMax != Vector2.one)
        chipLabelsStretched = false;
}
Assert("W23_hud_chip_row_wired",
    chipRowWired && chipLabelsStretched && chipBootHidden,
    "four chips panel-wired=" + chipRowWired
+ " labels stretch-child=" + chipLabelsStretched
+ " boot-hidden=" + chipBootHidden
+ " (receipt strip LIST/DEAL/DOWN/SAVED built, hidden until the stamp)");

// X1231 T016-d6 compose law: the receipt strip reads
// the guided-boot bargain back exactly (list 40 ->
// deal 35, down (3p+9)/10 = 11, saved 5) and the
// out-of-range index composes empty (no orphan chip
// can show stale copy).
var chipCompose = BargainViewHost.ComposeChipText(0, 40, 35, 11, 5) == "LIST 40 WAN"
    && BargainViewHost.ComposeChipText(1, 40, 35, 11, 5) == "DEAL 35 WAN"
    && BargainViewHost.ComposeChipText(2, 40, 35, 11, 5) == "DOWN 11 WAN"
    && BargainViewHost.ComposeChipText(3, 40, 35, 11, 5) == "SAVED 5 WAN"
    && BargainViewHost.ComposeChipText(4, 40, 35, 11, 5) == "";
Assert("W24_chip_receipt_compose_law", chipCompose,
    "guided receipt compose=" + chipCompose
+ " (LIST 40 WAN / DEAL 35 WAN / DOWN 11 WAN / SAVED 5 WAN, index 4 empty)");

```

```

// Real-host drive: a Wang-bound listing duel shows
// both busts; rebinding to a portrait-less agent
// hides the agent bust while the landlord stays on.
var bootPlayerActive = playerSeatFace != null && playerSeatFace.gameObject.activeSelf;
duelHost.StartListingBargain(120, 1, 1);
var wangPlayerActive = playerSeatFace != null && playerSeatFace.gameObject.activeSelf;
var wangAgentActive = agentSeatFace != null && agentSeatFace.gameObject.activeSelf;
duelHost.StartListingBargain(120, 3, 1);
var rebindAgentActive = agentSeatFace != null && agentSeatFace.gameObject.activeSelf;
var gateLaw = !BargainViewHost.AgentPortraitSeatVisible(-1)
    && !BargainViewHost.AgentPortraitSeatVisible(2)
    && !BargainViewHost.AgentPortraitSeatVisible(5)
    && BargainViewHost.AgentPortraitSeatVisible(1);
Assert("W19_agent_seat_gate_law",
    !bootPlayerActive && wangPlayerActive && wangAgentActive
    && !rebindAgentActive && gateLaw,
    "boot-hidden=" + !bootPlayerActive
    + " wangDuel player=" + wangPlayerActive + " agent=" + wangAgentActive
    + " rebindAgent3Shown=" + rebindAgentActive
    + " gate(-1,2,5 hidden;1 shown)=" + gateLaw
    + " (Tiezui Wang signs the duel face, portrait-less agents honestly hide)");

// X1229 T016-d5 segment law: the driven listing duel
// shows the chrome - the dialogue panel + accent
// strip enter with the room band at session start
// (RetireHookArt segment law, W21 freezes the swap).
var chromeActive = bubblePanelFace != null && bubblePanelFace.gameObject.activeSelf
    && duelAccentFace != null && duelAccentFace.gameObject.activeSelf;
Assert("W21_duel_chrome_segment_law", chromeActive,
    "driven duel bubble panel active=" + (bubblePanelFace != null && bubblePanelFace.gameObject.activeSelf)
    + " HUD accent active=" + (duelAccentFace != null && duelAccentFace.gameObject.activeSelf)
    + " (chrome enters with the room band at session start)");

// X1231 T016-d6 segment law: the receipt chips are
// settle-only chrome - the driven listing duel (and
// every rebind) keeps all four hidden; they show with
// the settle card at the stamp and clear on the next
// rebind (StartListingBargain sweep).
var chipsLeaked = false;
for (var i = 0; i < 4; i++)
{
    var chipFace = drt.Find("HudChip" + i);
    if (chipFace != null && chipFace.gameObject.activeSelf) chipsLeaked = true;
}
Assert("W25_chip_segment_law", !chipsLeaked,
    "driven duel + rebind chip leak=" + chipsLeaked
    + " (receipt chips stay hidden through the duel segment; only the stamp shows them)");

// X1236 T016-d9 settle rain-rent strip law (U088

```

```

// wave-6 judge debt: the collect line floated
// unanchored in the settle band): the strip carries
// the same panel chrome as the receipt chips, its
// label rides a stretch child (one-Graphic-per-GO
// law, X1192), composes through the rent desk row's
// single truth (ComposeCollectLabel), and stays
// boot-hidden with the settle segment.
var rentStripFace = drt.Find("SettleRentStrip");
var rentStripLabelRt = rentStripFace != null
    ? rentStripFace.Find("Label") as RectTransform : null;
var rentStripImg = rentStripFace != null ? rentStripFace.GetComponent<Image>() : null;
var rentStripTxt = rentStripLabelRt != null
    ? rentStripLabelRt.GetComponent<Text>() : null;
bool rentStripChrome = rentStripImg != null && panelArt != null
    && rentStripImg.sprite == panelArt
    && rentStripImg.type == Image.Type.Sliced
    && !rentStripImg.raycastTarget;
bool rentStripFaceTrue = rentStripTxt != null
    && rentStripTxt.text == RentViewHost.ComposeCollectLabel(0);
Assert("W28_settle_rent_strip_law",
    rentStripFace != null && !rentStripFace.gameObject.activeSelf
    && rentStripChrome && rentStripFaceTrue
    && rentStripLabelRt != null
    && rentStripLabelRt.anchorMin == Vector2.zero
    && rentStripLabelRt.anchorMax == Vector2.one,
    "strip boot-hidden=" + (rentStripFace != null && !rentStripFace.gameObject.activeSelf)
    + " panel-sliced=" + (rentStripImg != null && rentStripImg.sprite == panelArt
    && rentStripImg.type == Image.Type.Sliced)
    + " face=\"" + (rentStripTxt != null ? rentStripTxt.text : "n/a") + "\"
    + " label-stretch-child=" + (rentStripLabelRt != null
    && rentStripLabelRt.anchorMin == Vector2.zero
    && rentStripLabelRt.anchorMax == Vector2.one)
    + " (rain-rent line anchored to the settle band, single truth with the desk row)");
    }
}
finally
{
    // BargainCanvas is root-built (BuildUi parents it to
    // nothing), the outer finally sweeps it by name below.
}

// ---- W4 paint face: the wired faces must actually paint ----
RunPaintFace(canvasGo, cityArt != null && panelArt != null && btnArt != null);
}
catch (Exception e)
{
    _failed++;
    Lines.Add("[FAIL] W0_exception - " + e.Message);
}
finally

```

```

    {
        // ListingCanvas is built as a ROOT object (BuildUi creates it
        // parentless), so the sweep must cover it alongside the seat
        // and the rig or the next gate inherits a live canvas.
        foreach (var go in UnityEngine.Object.FindObjectsOfType<GameObject>())
            if (go.name == "ArtWireHostSeat" || go.name == "ArtWireRig" || go.name == "ListingCanvas"
                || go.name == "BargainWireHostSeat" || go.name == "BargainCanvas")
                UnityEngine.Object.DestroyImmediate(go);
    }
    WriteReport();
}

// Feed-style rig render (X1192 QG-ext pattern): adapt the real host
// canvas to a WorldSpace rig 1:1, then dense-probe the painted city
// band, panel field and primary button against the rig background.
static void RunPaintFace(GameObject canvasGo, bool artReady)
{
    if (!artReady)
    {
        Assert("W8_paint_face", false, "skipped: art not ready (catalog face red above)");
        return;
    }
    GameObject rig = null;
    RenderTexture rtTex = null;
    try
    {
        {
            rig = new GameObject("ArtWireRig");
            var camGo = new GameObject("Cam");
            camGo.transform.SetParent(rig.transform, false);
            var cam = camGo.AddComponent<Camera>();
            cam.orthographic = true;
            cam.orthographicSize = FrameH * 0.5f;
            cam.aspect = FrameW / (float)FrameH;
            cam.nearClipPlane = 0.1f;
            cam.farClipPlane = 500f;
            cam.transform.position = new Vector3(0f, 0f, -100f);
            cam.cullingMask = 1 << Uilayer;
            cam.clearFlags = CameraClearFlags.SolidColor;
            cam.backgroundColor = RigBg;
            cam.enabled = false;
            SetLayer(rig, Uilayer);
            SetLayer(canvasGo, Uilayer);

            var canvas = canvasGo.GetComponent<Canvas>();
            canvas.renderMode = RenderMode.WorldSpace;
            canvas.worldCamera = cam;
            canvas.planeDistance = 100f;
            var crt = (RectTransform)canvasGo.transform;
            crt.localPosition = Vector3.zero;
            crt.localRotation = Quaternion.identity;
        }
    }
    catch { }
}

```

(middle pages omitted)

pages 31 - 495 of 525 omitted per first-30 + last-30 excerpt rule.

```

    {
        _holdingsBadge.gameObject.SetActive(storageOpen);
        if (storageOpen) _holdingsBadge.text = ComposeHoldingsBadge(_ledger.Rows.Count);
    }
    if (_storageFace != null)
    {
        _storageFace.gameObject.SetActive(storageOpen);
        if (storageOpen) _storageFace.text = ComposeStorageFace(_ledger.Rows.Count, _ledger.LiquidWan);
    }
}

void RefreshCrisisFace()
{
    if (_crisisPopup == null || _iaa == null) return;
    var visible = CrisisFaceVisible(AdDayKey(), _crisisDeclinedDay, _dayKey, _iaa);
    if (visible && !_crisisPopup.activeSelf)
    {
        CrisisFaceShown = true;
        // T022-d1c: the bank call going live IS the SYS03 cash-flow
        // tight signal (ring-1 ch9 footprint - the pair the two real
        // sell actions complete).
        StoryRuntime.Feed(GateFeed.CashTight);
        // T012 slice B live trigger face: the distress copy names the
        // two outs - bank-grace ad or sell a house (U091 no dead end).
        _crisisFace.text = CopyTable.Get("crisis_show_title") + "\n"
            + CopyTable.Get("crisis_show_body");
    }
    _crisisPopup.SetActive(visible);
}

void SetDeskOpen(bool open)
{
    _panel.SetActive(open);
    _openButton.gameObject.SetActive(!open);
    if (open) RefreshCollectFace();
}

void OnOpenTapped()
{
    Taps++;
    SetDeskOpen(true);
}

// Rent rain: drain the pending pool to lifetime, fold the popup.
void OnCollectTapped()
{
    Taps++;
    CollectTaps++;
    LastCollectAmount = _ledger.Collect();
    if (_summaryPopup != null && _summaryPopup.activeSelf) _summaryPopup.SetActive(false);
}

```

```

        RefreshCollectFace();
    }

    void OnDoubleTapped()
    {
        Taps++;
        DoubleTaps++;
        if (!TryAcceptDoubleTap(_dayKey, _ledger)) return;
        RefreshCollectFace();
    }

    void OnSkipTapped()
    {
        Taps++;
        SkipTaps++;
        _declinedDay = _dayKey; // offer declined for the rest of today
        RefreshCollectFace();
    }

    // T008 slice-B: ad-gated double - the grant doubles the pending rain
    // through the ledger law and banks one lottery sign; the toast shows
    // the doubled gain (iaa_double_done face).
    void OnIaaDoubleTapped()
    {
        Taps++;
        IaaDoubleTaps++;
        if (!TryIaaDoubleTap(AdDayKey(), _dayKey, _iaa, _ledger)) return;
        if (_iaaToast != null)
            _iaaToast.text = ComposeIaaDoubleDone(_ledger.PendingCoins);
        RefreshCollectFace();
    }

    void OnIaaSkipTapped()
    {
        Taps++;
        IaaSkipTaps++;
        _iaaDeclinedDay = _dayKey; // ad pair declined for the rest of today
        RefreshCollectFace();
    }

    // Bank-grace rescue: the ad clears the distress flag and opens the
    // 30-minute grace window (T012 passes a real mortgage id later; S1
    // live play never raises the flag, so 0 is a dormant placeholder).
    void OnCrisisAcceptTapped()
    {
        Taps++;
        CrisisTaps++;
        // T012 slice B: the real distressed row id replaces the dormant 0
        // (0 stays when the ledger is clean - S1 live play never redlines).
        if (_mortgageDesk == null) _mortgageDesk = FindObjectOfType<MortgageViewHost>();
    }

```

```

        var mortgageId = _mortgageDesk != null
            ? MortgageViewHost.DistressedUnitId(_mortgageDesk.Sim) : 0;
        if (!TryRescueTap(AdDayKey(), _iaa, mortgageId, Time.time)) return;
        if (_crisisFace != null) _crisisFace.text = CopyTable.Get("iaa_rescue_done");
    }

    void OnCrisisRefuseTapped()
    {
        Taps++;
        CrisisRefuses++;
        _crisisDeclinedDay = _dayKey; // bank call dismissed for today
    }

    // Welcome-back face: the boot settle report lands here and the desk
    // opens itself so the revisit summary is the first thing seen.
    public void ShowWelcomeBack(OfflineReport report)
    {
        if (_summaryPopup == null) return;
        _summaryBody.text = _ledger.OfflineSummaryText(report);
        _summaryPopup.SetActive(true);
        PopupShown = true;
        SetDeskOpen(true);
    }
}

// ===== FILE: Assets\Game\Scripts\Runtime\Rent\TenantTable.cs =====
// G16 tenant roster (SYS05 RentRow tenant face). Requirements doc tenant table:
// 10 comedy tenants from the Round-Ring City roster; deterministic per-row
// assignment (zero RNG API), all visible copy via CopyTable EN/ZH faces.
using System.Collections.Generic;

namespace Sy.CrazyEstate
{
    public struct TenantDef
    {
        public int Id;
        public string NameKey;
        public string ReviewKey;
    }

    public static class TenantTable
    {
        public const int Count = 10;

        public static readonly TenantDef[] Tenants =
        {
            new TenantDef { Id = 1, NameKey = "tenant_name_01", ReviewKey = "tenant_review_01" },
            new TenantDef { Id = 2, NameKey = "tenant_name_02", ReviewKey = "tenant_review_02" },
            new TenantDef { Id = 3, NameKey = "tenant_name_03", ReviewKey = "tenant_review_03" },
            new TenantDef { Id = 4, NameKey = "tenant_name_04", ReviewKey = "tenant_review_04" },
        }
    }
}

```

```

    new TenantDef { Id = 5, NameKey = "tenant_name_05", ReviewKey = "tenant_review_05" },
    new TenantDef { Id = 6, NameKey = "tenant_name_06", ReviewKey = "tenant_review_06" },
    new TenantDef { Id = 7, NameKey = "tenant_name_07", ReviewKey = "tenant_review_07" },
    new TenantDef { Id = 8, NameKey = "tenant_name_08", ReviewKey = "tenant_review_08" },
    new TenantDef { Id = 9, NameKey = "tenant_name_09", ReviewKey = "tenant_review_09" },
    new TenantDef { Id = 10, NameKey = "tenant_name_10", ReviewKey = "tenant_review_10" },
};

public static TenantDef GetTenant(int id)
{
    for (var i = 0; i < Tenants.Length; i++)
        if (Tenants[i].Id == id) return Tenants[i];
    return Tenants[0];
}

// Deterministic tenant for a portfolio row: pure function of the listing
// id and the row ordinal (same buys -> same tenants on double run; the
// 7/11 stride keeps consecutive units from colliding on one tenant).
public static int TenantForRow(int rowOrdinal, int unitId)
{
    return ((unitId * 7 + rowOrdinal * 11) % Count) + 1;
}

public static List<string> Validate(List<string> problems)
{
    if (problems == null) problems = new List<string>();
    if (Tenants.Length != Count) problems.Add("tenant roster short of 10");
    var zh = CopyTableZh.Snapshot();
    for (var i = 0; i < Tenants.Length; i++)
    {
        var t = Tenants[i];
        if (!CopyTable.TryGetRaw(t.NameKey, out var n) || string.IsNullOrEmpty(n))
            problems.Add("tenant name key missing: " + t.Id);
        if (!CopyTable.TryGetRaw(t.ReviewKey, out var r) || string.IsNullOrEmpty(r))
            problems.Add("tenant review key missing: " + t.Id);
        if (!zh.ContainsKey(t.NameKey)) problems.Add("tenant zh name missing: " + t.Id);
        if (!zh.ContainsKey(t.ReviewKey)) problems.Add("tenant zh review missing: " + t.Id);
        for (var j = i + 1; j < Tenants.Length; j++)
            if (Tenants[j].Id == t.Id) problems.Add("tenant id duplicated: " + t.Id);
    }
    return problems;
}
}

// ===== FILE: Assets\Game\Scripts\Runtime\Story\ArcState.cs =====
// G16 ArcState ring-3 rival-pool arc-state layer (T022-c, M15 Oasis Story
// Almanac section 6 ring 3): gameplay rewrites the story. The six rival-arc
// flip rows of the almanac table ride the ring-1 PlayGate chapter walk and
// rewrite the middleman duel pool - Hike hangs his listings at a dive price
// with the x1.5 comedy cut window after ch7, then leaves the pool for the

```

```

// noodle-shop egg seat after ch11 (meeting him again is an egg, not a duel);
// Iron-mouth leaves for the BBQ stall at the ch8 window and returns at the
// ch12 window as the honest agent (small cut, zero trap talk); the lottery
// koi sister enters the pool at the act-3 ch7 window as the rookie seat
// Lucky Hand Jie; Lisa mounts her mentor word-of-mouth star after ch9. Each
// flip fires evt_arc_flip exactly once (table order, idempotent latch). The
// story read-data faces keep the copy honest: the ch11 "peer" verdict reads
// the REAL accumulated Hike duel count, the grandma-livestream viewer
// number reads the REAL total closed-deal count, the ch10 revisit roster
// reads the real TenantTable, and the tycoon annual report reuses SYS09 (no
// duplicate face here). Pure decision layer like PlayGate/DramaBuff: zero
// RNG, zero clocks, culture-invariant fingerprint; host wiring (duel feeds,
// dive-price listings, view consumption) lands in the host slice.
using System;
using System.Globalization;
using System.Text;

namespace Sy.CrazyEstate
{
    public sealed class ArcState
    {
        // Almanac section 6 ring-3 flip table (six rows, verbatim order).
        public const int FlipCount = 6;
        public const int FlipHikeDive = 0;           // Hike ch7: dive price + x1.5 cut window
        public const int FlipHikeNoodleExit = 1;     // Hike ch11+: pool exit -> noodle egg
        public const int FlipIronBbqExit = 2;        // Iron ch8 window: pool exit (BBQ stall)
        public const int FlipIronHonestReturn = 3;    // Iron ch12 window: re-entry, honest rival
        public const int FlipKoiRookieEnter = 4;     // Koi act-3 ch7 window: rookie seat enters
        public const int FlipLisaMentorStar = 5;     // Lisa ch9+: mentor word-of-mouth star
        public const string ArcFlipEventId = "evt_arc_flip";

        // Pack-frozen initial anchors (numeric pack section 10.2; the pack
        // owns the re-anchor after the bot sweep - no mid-round value edits).
        public const double HikeDiveCutFactor = 1.5; // comedy dive-price cut window
        public const int HikeDuelPeerAnchor = 3;     // ch11 "you treated me as a peer" verdict
        public const int LisaMentorStarAdd = 1;      // mentor word-of-mouth star

        // Codex agent seats (CodexRosterTable): 1 Iron, 2 Weepy, 3 Hike,
        // 4 Zen, 5 Lucky Hand Jie (rookie seat, ring-3 entry).
        public const int AgentIron = 1;
        public const int AgentWeepy = 2;
        public const int AgentHike = 3;
        public const int AgentZen = 4;
        public const int AgentKoi = 5;

        // Flip trigger chapters (almanac ring-3 table verbatim).
        static readonly int[] FlipChapter = { 7, 11, 8, 12, 7, 9 };

        readonly bool[] _flipped = new bool[FlipCount];
        int _flipFires;
    }
}

```

```

int _lastFlipOrdinalThisSync;
int _hikeDuels;
int _dealsClosed;

public int FlipFires { get { return _flipFires; } }
public int LastFlipOrdinalThisSync { get { return _lastFlipOrdinalThisSync; } }
public int HikeDuelCount { get { return _hikeDuels; } }
public int TotalDealsClosed { get { return _dealsClosed; } }
public bool IsFlipped(int flip) { return flip >= 0 && flip < FlipCount && _flipped[flip]; }

/// <summary>evt_arc_flip anchor: fires exactly once per flip row,
/// carrying the flip ordinal (the funnel reads which story arcs the
/// gameplay has already rewritten).</summary>
public event Action<int> OnArcFlip;

/// <summary>Apply every ring-3 flip whose trigger chapter the ring-1
/// gate has unlocked (table order, idempotent latch - re-syncing
/// re-fires nothing). Returns how many flips THIS sync applied
/// (0 = nothing new; a zero-chapter player keeps the bare pool).</summary>
public int SyncFromGate(PlayGate gate)
{
    _lastFlipOrdinalThisSync = 0;
    var applied = 0;
    var unlocked = gate.UnlockedChapter;
    for (var f = 0; f < FlipCount; f++)
    {
        if (_flipped[f] || FlipChapter[f] > unlocked) continue;
        _flipped[f] = true;
        applied++;
        _lastFlipOrdinalThisSync = f + 1;
        _flipFires++;
        OnArcFlip?.Invoke(f);
    }
    return applied;
}

// ---- rival-pool query faces (identity until the flip lands) ----

/// <summary>Duel-pool membership: the rotation draws only from the
/// live pool. Iron is in until the ch8 BBQ exit and returns at the
/// ch12 honest window; Hike is in until his ch11 noodle exit; the
/// koi rookie seat is out until her ch7 window; Weepy and Zen never
/// flip (pool evergreen seats).</summary>
public bool IsAgentInPool(int agentId)
{
    switch (agentId)
    {
        case AgentIron:
            return !_flipped[FlipIronBbqExit] || _flipped[FlipIronHonestReturn];
        case AgentWeepy:

```

```

        case AgentZen:
            return true;
        case AgentHike:
            return !_flipped[FlipHikeNoodleExit];
        case AgentKoi:
            return _flipped[FlipKoiRookieEnter];
        default:
            return false;
    }
}

/// <summary>Hike's dive-price comedy egg: after ch7 his listings hang
/// a dive price and the cut window widens x1.5 (misery as comedy) -
/// only while he is still dueling in the pool.</summary>
public bool HikeDivePricingActive()
{
    return _flipped[FlipHikeDive] && !_flipped[FlipHikeNoodleExit];
}

public double HikeCutWindowFactor()
{
    return HikeDivePricingActive() ? HikeDiveCutFactor : 1.0;
}

/// <summary>Meeting Hike after his ch11 exit is the noodle-shop egg,
/// never a duel (his pool seat is gone for good).</summary>
public bool HikeNoodleEggActive() { return _flipped[FlipHikeNoodleExit]; }

/// <summary>Iron's honest re-entry state (ch12 window): small cut,
/// zero trap talk - the agent who learned to tell the truth.</summary>
public bool IronMouthHonestActive() { return _flipped[FlipIronHonestReturn]; }

/// <summary>Lisa's mentor word-of-mouth star (ch9+ agent-line gain,
/// the sweeping-monk cameo promise cashed - mirrors the DramaBuff
/// ch9 star face on the license line).</summary>
public int LisaMentorStarBonus() { return _flipped[FlipLisaMentorStar] ? LisaMentorStarAdd : 0; }

// ---- story read-data faces (copy variables read REAL save values;
// the annual-report face reuses SYS09 - no duplicate face here) ----

/// <summary>Host feeds one closed deal (evt_deal_close): the grandma
/// livestream viewer number reads this REAL total - views are earned,
/// never invented.</summary>
public void FeedDealClosed() { _dealsClosed++; }

/// <summary>Host feeds one Hike duel: the ch11 "only you treated me
/// as a peer" verdict reads this REAL accumulated count (the more you
/// haggled, the warmer the noodle bowl).</summary>
public void FeedHikeDuel() { _hikeDuels++; }

```

```

public bool HikeDuelPeerVerdictMet() { return _hikeDuels >= HikeDuelPeerAnchor; }

/// <summary>Ch10 revisit-roster window: the roster itself reads the
/// REAL TenantTable at the view slice (whoever complains the most
/// gets the biggest comeback scene) - this face only gates the
/// chapter window, the read-data law keeps TenantTable the single
/// legal source.</summary>
public bool RevisitRosterReady(PlayGate gate) { return gate.IsUnlocked(10); }

public string Fingerprint()
{
    var ci = CultureInfo.InvariantCulture;
    var sb = new StringBuilder();
    sb.Append("F");
    for (var f = 0; f < FlipCount; f++) sb.Append(_flipped[f] ? 1 : 0);
    sb.Append(":f").Append(_flipFires);
    sb.Append(":d").Append(_hikeDuels);
    sb.Append(":c").Append(_dealsClosed);
    sb.Append(":p");
    for (var a = 1; a <= AgentKoi; a++) sb.Append(IsAgentInPool(a) ? 1 : 0);
    sb.Append(":w").Append(HikeCutWindowFactor().ToString("0.##", ci));
    sb.Append(":n").Append(HikeNoodleEggActive() ? 1 : 0);
    sb.Append(":h").Append(IronMouthHonestActive() ? 1 : 0);
    sb.Append(":m").Append(LisaMentorStarBonus());
    return sb.ToString();
}
}

// ===== FILE: Assets_Game\Scripts\Runtime\Story\ArcTheater.cs =====
// G16 rival-arc act engine + warm-chain late-night theater (T021-b, M15
// Oasis Story Almanac sections 2 and 4). The four rival NPCs (Raise-Bro the
// overnight price-hiker, the Spec-Squad Commander, Top-Seller Lisa, Grandma)
// mount one act line on every story chapter (the twelve chapters quarter into
// four acts), and the warm-chain mini-theater plays its two-beat structure
// (absurd setup -> warm landing) one beat per face. Pure decision layer, zero
// RNG, culture-invariant fingerprints for gate double runs. Redline-11 law
// faces shared with SkylineShow: never play inside the opening window, never
// preempt a running face, every face strictly shorter than the event-popup
// auto-hide (no stacked faces).
using System.Globalization;
using System.Text;

namespace Sy.CrazyEstate
{
    public enum ArcPhase { Idle = 0, Rise = 1, Hold = 2, Fall = 3 }

    public sealed class ArcShow
    {
        // Rival cast (M15 section 2).
        public const int Hike = 0;
    }
}

```

```

public const int Squad = 1;
public const int Lisa = 2;
public const int Granny = 3;
public static readonly int[] RivalIds = { Hike, Squad, Lisa, Granny };

public const float RiseSec = 0.6f;
public const float HoldSec = 2.4f; // the one readable beat
public const float FallSec = 0.8f;
public const float TotalSec = RiseSec + HoldSec + FallSec;
public const float OpeningFaceSec = 10f; // redline-11 opening window anchor
public const int ChapterCount = 12;

// Story quarters: twelve chapters -> four acts (act = chapter quarter).
public static int ActForChapter(int chapter)
{
    if (chapter < 1 || chapter > ChapterCount) return 0;
    return (chapter - 1) / 3 + 1;
}

// Act lines each rival carries (section 2 scripts: Squad joins in
// act 2, Lisa skips act 2 - the others span all four acts).
public static int ActsFor(int rival)
{
    return rival == Hike || rival == Granny ? 4 : 3;
}

// The act a rival mounts on a chapter (0 = sits this quarter out).
public static int ActMountedOn(int rival, int chapter)
{
    var act = ActForChapter(chapter);
    if (act == 0 || rival < 0 || rival > 3) return 0;
    if (rival == Hike || rival == Granny) return act;
    if (rival == Squad) return act == 1 ? 0 : act - 1;
    return act == 1 ? 1 : act == 2 ? 0 : act - 1;
}

public static string KeyFor(int rival, int act)
{
    var name = rival == Hike ? "hike" : rival == Squad ? "squad"
        : rival == Lisa ? "lisa" : "granny";
    return "arc_" + name + "_" + act;
}

ArcPhase _phase;
float _phaseSec;
string _key;

public ArcPhase Phase { get { return _phase; } }
public bool Playing { get { return _phase != ArcPhase.Idle; } }
public string Key { get { return _key; } }

```

```

// One act line per face: outside the opening window, never
// preempting, and the copy key must resolve on the table.
public bool BeginAct(int rival, int chapter, float hostSec)
{
    var act = ActMountedOn(rival, chapter);
    if (Playing || act == 0) return false;
    if (hostSec < OpeningFaceSec) return false;
    if (!CopyTable.TryGetRaw(KeyFor(rival, act), out var line) || string.IsNullOrEmpty(line)) return false;
    _key = KeyFor(rival, act);
    _phase = ArcPhase.Rise;
    _phaseSec = 0f;
    return true;
}

public ArcPhase Tick(float dt)
{
    if (_phase == ArcPhase.Idle || dt <= 0f) return _phase;
    _phaseSec += dt;
    var span = PhaseSpan(_phase);
    if (_phaseSec >= span)
    {
        _phaseSec -= span;
        _phase = _phase == ArcPhase.Rise ? ArcPhase.Hold
            : _phase == ArcPhase.Hold ? ArcPhase.Fall
            : ArcPhase.Idle;
        if (_phase == ArcPhase.Idle) { _phaseSec = 0f; _key = null; }
    }
    return _phase;
}

static float PhaseSpan(ArcPhase phase)
{
    return phase == ArcPhase.Rise ? RiseSec : phase == ArcPhase.Hold ? HoldSec : FallSec;
}

public static bool FaceShorterThanPopupAutoHide()
{
    return TotalSec < EventPopupHost.AutoHideSec;
}

public string Fingerprint()
{
    var sb = new StringBuilder();
    sb.Append("a").Append("p").Append((int)_phase);
    sb.Append(":t").Append(((int)(_phaseSec * 1000)).ToString("D6", CultureInfo.InvariantCulture));
    sb.Append(":k").Append(_key == null ? "-" : _key);
    return sb.ToString();
}
}

```

```

public sealed class TheaterShow
{
    // Warm chains (M15 section 4 late-night mini-theater): W01 the old
    // locust tree (chapters 3 and 12), W02 the south-facing room (ch 9),
    // W03 one bowl between rivals (ch 11).
    public const int W01 = 1;
    public const int W02 = 2;
    public const int W03 = 3;
    public const int BeatsPerChain = 2; // absurd setup -> warm landing

    public const float RiseSec = 0.6f;
    public const float HoldSec = 2.4f;
    public const float FallSec = 0.8f;
    public const float TotalBeatSec = RiseSec + HoldSec + FallSec;
    public const float OpeningFaceSec = 10f;

    // The chain that mounts on a chapter (section 4 event hooks; 0 = none).
    public static int ChainForChapter(int chapter)
    {
        return chapter == 3 || chapter == 12 ? W01
            : chapter == 9 ? W02
            : chapter == 11 ? W03
            : 0;
    }

    public static string TitleKeyFor(int chain)
    {
        return "w_theater_w0" + chain + "_title";
    }

    public static string BeatKeyFor(int chain, int beat)
    {
        return "w_theater_w0" + chain + "_beat" + beat;
    }

    ArcPhase _phase;
    float _phaseSec;
    int _chain;
    int _beat;

    public ArcPhase Phase { get { return _phase; } }
    public bool Playing { get { return _phase != ArcPhase.Idle; } }
    public int Chain { get { return _chain; } }
    public int Beat { get { return _beat; } }

    // One beat per face: the view starts the next beat only after the
    // current face ends (no stacked faces). Same redline-11 guards.
    public bool BeginBeat(int chain, int beat, float hostSec)
    {

```

```

        if (Playing) return false;
        if (chain < W01 || chain > W03 || beat < 1 || beat > BeatsPerChain) return false;
        if (hostSec < OpeningFaceSec) return false;
        if (!CopyTable.TryGetRaw(BeatKeyFor(chain, beat), out var line) || string.IsNullOrEmpty(line)) return false;
        _chain = chain;
        _beat = beat;
        _phase = ArcPhase.Rise;
        _phaseSec = 0f;
        return true;
    }

    public ArcPhase Tick(float dt)
    {
        if (_phase == ArcPhase.Idle || dt <= 0f) return _phase;
        _phaseSec += dt;
        var span = _phase == ArcPhase.Rise ? RiseSec : _phase == ArcPhase.Hold ? HoldSec : FallSec;
        if (_phaseSec >= span)
        {
            _phaseSec -= span;
            _phase = _phase == ArcPhase.Rise ? ArcPhase.Hold
                : _phase == ArcPhase.Hold ? ArcPhase.Fall
                : ArcPhase.Idle;
            if (_phase == ArcPhase.Idle) { _phaseSec = 0f; _chain = 0; _beat = 0; }
        }
        return _phase;
    }

    public static bool BeatShorterThanPopupAutoHide()
    {
        return TotalBeatSec < EventPopupHost.AutoHideSec;
    }

    public string Fingerprint()
    {
        var sb = new StringBuilder();
        sb.Append("w").Append("p").Append((int)_phase);
        sb.Append(":t").Append(((int)(_phaseSec * 1000)).ToString("D6", CultureInfo.InvariantCulture));
        sb.Append(":c").Append(_chain).Append(":b").Append(_beat);
        return sb.ToString();
    }
}

// ===== FILE: Assets\_Game\Scripts\Runtime\Story\DramaBuff.cs =====
// G16 DramaBuff ring-2 gain chain (T022-b, M15 Oasis Story Almanac section
// 6): every chapter end mounts one flavor gain the story cast gifts to the
// gameplay (ch1 grandma's video check-in -> bargain window +10%, ch2 the
// landlady coaches rent collection -> first-month rent +20%, ..., ch11 the
// noodle bowl -> every agent talk cut +1 wan, ch12 the city crown -> finale
// halo). The chain is STRICTLY OPTIONAL comfort (the gain law): a player who
// never advances a chapter keeps the full bare baseline on every face - the

```

```

// T013 bot sweep proves the no-buff baseline passes, a gain is never a gate
// (U091 zero-harm). Mounts ride the ring-1 PlayGate chapter walk (one gain
// per chapter, strict linear order) and fire evt_drama_buff exactly once per
// gain (the funnel reads "players chase the story for the gains"). Pure
// decision layer like PlayGate: zero RNG, zero clocks, culture-invariant
// fingerprint; effect consumers wire in the host slice - this layer owns the
// mount state and the query faces only.
using System;
using System.Globalization;
using System.Text;

namespace Sy.CrazyEstate
{
    public sealed class DramaBuff
    {
        public const int GainCount = 12;
        public const string GainMountEventId = "evt_drama_buff";

        // Pack-frozen initial anchors (numeric pack section 10.1; the pack
        // owns the re-anchor after the bot sweep - no mid-round value edits).
        public const double BargainWindowPct = 10.0; // ch1 grandma checks in
        public const double FirstMonthRentPct = 20.0; // ch2 rent coaching
        public const double DemolitionWeightAdd = 1.0; // ch3 tea-party intel
        public const int LotteryToleranceBeats = 1; // ch4 koi mantra
        public const double LicenseGiftPct = 3.0; // ch5 squad joining gift (flat first freeze)
        public const double MortgageDecayAdd = 0.05; // ch7 the koi's milk tea
        public const double RevivalSpeedFactor = 1.5; // ch8 lamp-keeper mutual aid
        public const double ShareExposureAdd = 1.0; // ch9 grandma livestream debut
        public const int ReputationStarAdd = 1; // ch9 word-of-mouth star
        public const double RentRainQuotaAdd = 1.0; // ch10 tenant revisit squad
        public const double AgentCutBonusWan = 1.0; // ch11 the noodle bowl
        public const double FinalHaloPct = 2.0; // ch12 the city crown halo

        int _mounted; // gains mounted 1.._mounted (linear, never skips)
        int _mountFires;
        int _lastMountThisSync;

        public int MountedCount { get { return _mounted; } }
        public int MountFires { get { return _mountFires; } }
        public int LastMountedThisSync { get { return _lastMountThisSync; } }
        public bool IsMounted(int chapter) { return chapter >= 1 && chapter <= _mounted; }

        /// <summary>evt_drama_buff anchor: fires exactly once per gain,
        /// carrying the chapter whose story ending gifted it.</summary>
        public event Action<int> OnGainMounted;

        /// <summary>Mount every gain whose chapter the ring-1 gate has
        /// unlocked. Idempotent latch: re-syncing the same walk re-fires
        /// nothing. Returns how many gains THIS sync mounted (0 = nothing
        /// new - the zero-chapter player stays on the bare baseline).</summary>

```

```

public int SyncFromGate(PlayGate gate)
{
    _lastMountThisSync = 0;
    var mountedThisSync = 0;
    var target = gate.UnlockedChapter;
    if (target > GainCount) target = GainCount;
    while (_mounted < target)
    {
        _mounted++;
        mountedThisSync++;
        _lastMountThisSync = _mounted;
        _mountFires++;
        OnGainMounted?.Invoke(_mounted);
    }
    return mountedThisSync;
}

// ---- effect query faces: identity until the gifting chapter lands ----
// (every face returns the bare-baseline value when the gain is not
// mounted, so the T013 numerics sweep is structurally passable with
// zero chapter progress - comfort only, never a required gate).

public double BargainWindowBonusPct() { return IsMounted(1) ? BargainWindowPct : 0.0; }
public double FirstMonthRentBonusPct() { return IsMounted(2) ? FirstMonthRentPct : 0.0; }
public double DemolitionEventWeightAdd() { return IsMounted(3) ? DemolitionWeightAdd : 0.0; }
public int LotteryToleranceBonusBeats() { return IsMounted(4) ? LotteryToleranceBeats : 0; }
public double LicenseGiftBonusPct(int licenseTier) { return IsMounted(5) ? LicenseGiftPct : 0.0; }
public bool MarketDownRentFloorActive() { return IsMounted(6); }
public double MortgagePressureDecayAdd() { return IsMounted(7) ? MortgageDecayAdd : 0.0; }
public double RevivalSpeedBonusFactor() { return IsMounted(8) ? RevivalSpeedFactor : 1.0; }
public double ShareExposureBonus() { return IsMounted(9) ? ShareExposureAdd : 0.0; }
public int ReputationStarBonus() { return IsMounted(9) ? ReputationStarAdd : 0; }
public double RentRainQuotaBonus() { return IsMounted(10) ? RentRainQuotaAdd : 0.0; }
public double AgentTalkCutBonusWan() { return IsMounted(11) ? AgentCutBonusWan : 0.0; }
public double FinalHaloBonusPct() { return IsMounted(12) ? FinalHaloPct : 0.0; }

public string Fingerprint()
{
    var ci = CultureInfo.InvariantCulture;
    var sb = new StringBuilder();
    sb.Append("M").Append(_mounted);
    sb.Append(":f").Append(_mountFires);
    sb.Append(":w").Append(BargainWindowBonusPct().ToString("0.##", ci));
    sb.Append(":r").Append(FirstMonthRentBonusPct().ToString("0.##", ci));
    sb.Append(":d").Append(DemolitionEventWeightAdd().ToString("0.##", ci));
    sb.Append(":l").Append(LotteryToleranceBonusBeats());
    sb.Append(":g").Append(LicenseGiftBonusPct(1).ToString("0.##", ci));
    sb.Append(":mk").Append(MarketDownRentFloorActive() ? 1 : 0);
    sb.Append(":p").Append(MortgagePressureDecayAdd().ToString("0.##", ci));
    sb.Append(":v").Append(RevivalSpeedBonusFactor().ToString("0.##", ci));
}

```

```

        sb.Append("s").Append(ShareExposureBonus().ToString("0.##", ci)).Append(ReputationStarBonus());
        sb.Append("q").Append(RentRainQuotaBonus().ToString("0.##", ci));
        sb.Append("a").Append(AgentTalkCutBonusWan().ToString("0.##", ci));
        sb.Append("h").Append(FinalHaloBonusPct().ToString("0.##", ci));
        return sb.ToString();
    }
}

}

// ===== FILE: Assets_Game\Scripts\Runtime\Story\PlayGate.cs =====
// G16 PlayGate chapter-gate data layer (T022-a ring 1, M15 Oasis Story Almanac
// section 6): the twelve story chapters unlock ONLY on real gameplay milestones
// - never a time wall. The gate latches the player's footprint facts (first
// deal, first reno, first tenant, demolition, lottery, wealth, first pot,
// crisis, H07 holding, cash-tight sells, tenant scale, five-line finale, city
// crown), walks the chapters in strict linear order (an unmet gate blocks every
// later chapter - no skipping), and fires evt_chapter_unlock exactly once per
// chapter. Pure decision layer, zero RNG, culture-invariant fingerprint; the
// View slice consumes UnlockedChapter / LastUnlockedThisFeed to mount the
// ArcShow act seats (X1114 mount points).
using System;
using System.Globalization;
using System.Text;

namespace Sy.CrazyEstate
{
    /// <summary>Gameplay milestone feeds (SYS anchors, almanac section 6 ring-1 table).</summary>
    public enum GateFeed
    {
        DealClosed = 0,          // evt_deal_close: intPayload = listing id
        RenoCompleted = 1,       // SYS04 first renovation completed
        TenantMovedIn = 2,       // SYS05 first tenant moved in
        DemolitionEvent = 3,     // evt_demolition (E02 gold rain)
        LotteryEntered = 4,      // SYS08 first lottery draw entered
        NetWorthSnapshot = 5,    // asset snapshot: payload = net worth (wan)
        ResaleSettled = 6,       // resale settle: payload = single-flip profit (wan)
        MortgageCrisis = 7,      // SYS07 first mortgage-crisis pressure event
        HoldingH07 = 8,          // SYS01 holding H07 Future-City pre-sale
        CashTight = 9,           // SYS03 cash-flow tight signal
        HouseSold = 10,          // SYS03 sell action (count feed)
        TenantsHoused = 11,      // intPayload = current tenants-housed count
        PackageLicense = 12,     // LIC02 landlord license unlocked
        LinesAtFinale = 13,     // intPayload = five-line finale-tier count (M14)
    }

    public sealed class PlayGate
    {
        public const int ChapterCount = 12;
        // Pack-frozen anchors (G16 numeric pack section 10): the ch5 wealth
        // line re-anchors to the LIC04 family verdict (X-batch bot sweep) -
        // the 400-wan boot grant crossed the old 90-wan anchor on the very

```

```

// first month beat (a boot-grant trivial unlock, the X1153 debt);
// 600 wan sits one growth milestone above the boot grant. The ch6
// first-pot anchor is an initial freeze - the bot sweep re-anchors
// per the standing pack law (no mid-round value edits).
public const double NetWorthGateWan = 600.0;
public const double FirstPotProfitWan = 10.0;
public const int Ch9SellCountGate = 2;
public const int Ch10TenantsGate = 5;
public const int Ch11LinesGate = 2;
// Identity anchors use the ListingTable id space (1-based, H01=1..H10=10
// - the same ids BargainViewHost.BoundListingId carries into FeedDealClosed).
public const int CityCrownListingId = 10; // H10 the world crown (ListingTable Id)
public const int FutureCityListingId = 7; // H07 the future city (SYS01 pre-sale dream)
public const string ChapterUnlockEventId = "evt_chapter_unlock";

// footprint latches: player facts, monotone where numeric
bool _dealClosed;
bool _dealClosedCityCrown;
bool _renoDone;
bool _tenantIn;
bool _demolitionSeen;
bool _lotteryEntered;
double _netWorthWanMax;
double _resaleProfitWanMax;
bool _mortgageCrisisSeen;
bool _holdingH07;
bool _cashTight;
int _sellCount;
int _tenantsHousedMax;
bool _packagelicense;
int _linesAtFinaleMax;

int _unlocked;
int _lastUnlockThisFeed;
int _unlockFires;

public int UnlockedChapter { get { return _unlocked; } }
public int LastUnlockedThisFeed { get { return _lastUnlockThisFeed; } }
public int UnlockFires { get { return _unlockFires; } }
public bool IsUnlocked(int chapter) { return chapter >= 1 && chapter <= _unlocked; }

/// <summary>evt_chapter_unlock anchor: fires exactly once per chapter,
/// carrying the unlocked chapter number (the funnel reads "where the
/// player got to = where the story got to").</summary>
public event Action<int> OnChapterUnlocked;

/// <summary>Feed one gameplay milestone (latch update + gate walk). The
/// return mirrors LastUnlockedThisFeed (highest chapter THIS feed
/// unlocked; 0 = the armed gate still waits - zero dead ends, keep
/// playing the core loop).</summary>

```

```

public int Feed(GateFeed feed, double payload = 0.0, int intPayload = 0)
{
    _lastUnlockThisFeed = 0;
    switch (feed)
    {
        case GateFeed.DealClosed:
            _dealClosed = true;
            if (intPayload == CityCrownListingId) _dealClosedCityCrown = true;
            break;
        case GateFeed.RenoCompleted: _renoDone = true; break;
        case GateFeed.TenantMovedIn: _tenantIn = true; break;
        case GateFeed.DemolitionEvent: _demolitionSeen = true; break;
        case GateFeed.LotteryEntered: _lotteryEntered = true; break;
        case GateFeed.NetWorthSnapshot:
            if (payload > _netWorthWanMax) _netWorthWanMax = payload;
            break;
        case GateFeed.ResaleSettled:
            if (payload > _resaleProfitWanMax) _resaleProfitWanMax = payload;
            break;
        case GateFeed.MortgageCrisis: _mortgageCrisisSeen = true; break;
        case GateFeed.HoldingH07: _holdingH07 = true; break;
        case GateFeed.CashTight: _cashTight = true; break;
        case GateFeed.HouseSold: _sellCount++; break;
        case GateFeed.TenantsHoused:
            if (intPayload > _tenantsHousedMax) _tenantsHousedMax = intPayload;
            break;
        case GateFeed.PackageLicense: _packageLicense = true; break;
        case GateFeed.LinesAtFinale:
            if (intPayload > _linesAtFinaleMax) _linesAtFinaleMax = intPayload;
            break;
    }
    WalkGates();
    return _lastUnlockThisFeed;
}

/// <summary>Walk the armed gate only: chapters advance strictly in
/// order, each chapter only after every previous gate hit (a deep
/// footprint can cascade several chapters in one feed, but the order
/// never skips).</summary>
void WalkGates()
{
    while (_unlocked < ChapterCount && GateMet(_unlocked + 1))
    {
        _unlocked++;
        _lastUnlockThisFeed = _unlocked;
        _unlockFires++;
        OnChapterUnlocked?.Invoke(_unlocked);
    }
}

```

```

/// <summary>The unlock condition of a chapter (almanac section 6
/// ring-1 gate table). Every condition is a latched gameplay footprint
/// - the gate never reads a clock, so no time wall can exist.</summary>
public bool GateMet(int chapter)
{
    switch (chapter)
    {
        case 1: return _dealClosed;
        case 2: return _renoDone && _tenantIn;
        case 3: return _demolitionSeen;
        case 4: return _lotteryEntered;
        case 5: return _netWorthWanMax >= NetWorthGateWan;
        case 6: return _resaleProfitWanMax >= FirstPotProfitWan;
        case 7: return _mortgageCrisisSeen;
        case 8: return _holdingH07;
        case 9: return _cashTight && _sellCount >= Ch9SellCountGate;
        case 10: return _tenantsHousedMax >= Ch10TenantsGate || _packageLicense;
        case 11: return _linesAtFinaleMax >= Ch11LinesGate;
        case 12: return _dealClosedCityCrown;
        default: return false;
    }
}

public string Fingerprint()
{
    var sb = new StringBuilder();
    sb.Append("g").Append(_unlocked);
    sb.Append(":f").Append(_unlockFires);
    sb.Append(":d")
        .Append(_dealClosed ? 1 : 0).Append(_dealClosedCityCrown ? 1 : 0)
        .Append(_renoDone ? 1 : 0).Append(_tenantIn ? 1 : 0)
        .Append(_demolitionSeen ? 1 : 0).Append(_lotteryEntered ? 1 : 0)
        .Append(_mortgageCrisisSeen ? 1 : 0).Append(_holdingH07 ? 1 : 0)
        .Append(_cashTight ? 1 : 0).Append(_packageLicense ? 1 : 0);
    sb.Append(":s").Append(_sellCount);
    sb.Append(":n").Append(_netWorthWanMax.ToString("0.##", CultureInfo.InvariantCulture));
    sb.Append(":r").Append(_resaleProfitWanMax.ToString("0.##", CultureInfo.InvariantCulture));
    sb.Append(":t").Append(_tenantsHousedMax);
    sb.Append(":l").Append(_linesAtFinaleMax);
    return sb.ToString();
}
}

// ===== FILE: Assets\Game\Scripts\Runtime\Story\StoryFeed.cs =====
// G16 StoryFeed read-model (T022-d3 slice a, M15 Oasis Story Almanac
// section 6, R185 feed two-shape law): the footprint-feed READ layer between
// the recorded story event stream and the future View faces - a BitLife-style
// vertical event stream (the story beats the player actually lived, newest on
// top, mapped verbatim from StoryRuntime.Stream: chapter-gate unlocks,
// comfort-gain mounts, rival-pool arc flips) plus the travel-frog-style

```

```

// album collection page (twelve chapter cards, the six rare arc-flip
// points, one collection progress line - every total rides the ring
// constants, never a hand-packed number). The model reads recorded history
// ONLY (the stream and the hub mounts) - it never re-derives a footprint
// and never invents one. The View slice mounts these faces at the
// chapter-gate sidebar / story-page timeline (feed position law: never a
// main-interface seat). Pure decision layer like the rings: zero RNG, zero
// clocks, culture-invariant fingerprint.
using System;
using System.Collections.Generic;
using System.Text;

namespace Sy.CrazyEstate
{
    /// <summary>Which ring anchor one feed row came from.</summary>
    public enum StoryFeedRowKind
    {
        ChapterUnlock = 0, // evt_chapter_unlock: Chapter = unlocked chapter
        GainMount = 1,     // evt_drama_buff: Chapter = the gifting chapter
        ArcFlip = 2        // evt_arc_flip: FlipOrdinal = flip ordinal
    }

    /// <summary>One vertical-feed row (BitLife-style stream entry), mapped
    /// verbatim from one StoryEventRecord - the View mounts its faces on
    /// these fields, the model adds no text of its own (faces come with the
    /// View slice).</summary>
    public sealed class StoryFeedRow
    {
        public StoryFeedRowKind Kind;
        public int Seq;           // stream order, 1-based (latest = biggest)
        public int Chapter;       // unlock / gain chapter (0 for arc flips)
        public int FlipOrdinal;   // arc-flip ordinal (0 for unlock / gain)
    }

    public static class StoryFeed
    {
        // Collectible totals ride the ring constants - structural counts,
        // not numeric-pack anchors.
        public static int ChapterTotal { get { return PlayGate.ChapterCount; } }
        public static int GainTotal { get { return DramaBuff.GainCount; } }
        public static int RareFlipTotal { get { return ArcState.FlipCount; } }

        /// <summary>Every collectible story moment the album tracks:
        /// chapter unlocks + gain mounts + rare arc flips.</summary>
        public static int MomentTotal
        {
            get { return ChapterTotal + GainTotal + RareFlipTotal; }
        }

        /// <summary>The vertical-feed rows, NEWEST FIRST (the feed shows the

```

```

/// latest lived beat on top), mapped verbatim from the recorded
/// stream - history as it happened, never re-derived.</summary>
public static List<StoryFeedRow> Rows()
{
    var stream = StoryRuntime.Stream;
    var rows = new List<StoryFeedRow>(stream.Count);
    for (var i = stream.Count - 1; i >= 0; i--) rows.Add(MapRow(stream[i]));
    return rows;
}

/// <summary>Feed length (one row per recorded story event).</summary>
public static int RowCount { get { return StoryRuntime.Stream.Count; } }

// ---- album collection page (travel-frog style) ----

/// <summary>One album card: has this chapter gate been collected
/// (unlocked) yet? Out-of-range chapters are never collected.</summary>
public static bool AlbumChapterUnlocked(int chapter)
{
    return StoryRuntime.Gate.IsUnlocked(chapter);
}

/// <summary>Album progress line: chapters collected so far.</summary>
public static int AlbumUnlockedCount
{
    get { return StoryRuntime.Gate.UnlockedChapter; }
}

/// <summary>Rare points collected: distinct arc flips recorded in
/// the stream (the single-fire law makes stream count = distinct).</summary>
public static int RareFlipCount
{
    get
    {
        var seen = new bool[ArcState.FlipCount];
        var count = 0;
        foreach (var e in StoryRuntime.Stream)
        {
            if (e.EventId != ArcState.ArcFlipEventId) continue;
            if (e.Payload < 0 || e.Payload >= ArcState.FlipCount) continue;
            if (seen[e.Payload]) continue;
            seen[e.Payload] = true;
            count++;
        }
        return count;
    }
}

/// <summary>Album collection progress: story moments lived so far
/// (every stream record is one collected moment).</summary>

```

```

    public static int CollectedMoments { get { return RowCount; } }

    /// <summary>Feed-model fingerprint: every row verbatim plus the
    /// album faces - the CH19 double-run determinism face.</summary>
    public static string FeedFingerprint()
    {
        var sb = new StringBuilder();
        foreach (var r in Rows())
            sb.Append((int)r.Kind).Append('.').Append(r.Seq).Append('.')
              .Append(r.Chapter).Append('.').Append(r.FlipOrdinal).Append(';');
        sb.Append('|')
          .Append("u").Append(AlbumUnlockedCount)
          .Append("f").Append(RareFlipCount)
          .Append("m").Append(CollectedMoments)
          .Append("t").Append(MomentTotal);
        return sb.ToString();
    }

    static StoryFeedRow MapRow(StoryEventRecord e)
    {
        var row = new StoryFeedRow();
        row.Seq = e.Seq;
        if (e.EventId == PlayGate.ChapterUnlockEventId)
        {
            row.Kind = StoryFeedRowKind.ChapterUnlock;
            row.Chapter = e.Payload;
        }
        else if (e.EventId == DramaBuff.GainMountEventId)
        {
            row.Kind = StoryFeedRowKind.GainMount;
            row.Chapter = e.Payload;
        }
        else
        {
            row.Kind = StoryFeedRowKind.ArcFlip;
            row.FlipOrdinal = e.Payload;
        }
        return row;
    }
}

}

// ===== FILE: Assets\_Game\Scripts\Runtime\Story\StoryFeedViewHost.cs =====
// G16 story feed view host (T022-d3 slice b, M15 Oasis Story Almanac
// section 6 / R185 feed two-shape + position law): the READ face of the
// footprint feed - a chapter-gate sidebar strip on the right edge (the
// gate progress face, tap to open) and the story page it opens: the
// BitLife-style vertical timeline (every recorded row one face line,
// newest on top) plus the travel-frog-style collection album line.
// Position law: the whole face rides ONE canvas at order 2, strictly
// under every wall and desk face (3-11), the popup (12) and the act

```

```

// show (13) - an edge/reading seat that can never preempt a gameplay
// face, never a main-interface position; the page opens ONLY from the
// sidebar tab tap (passive read model, the host never feeds events).
// Faces compose from CopyTable through the pure decision layer below
// (single truth, shared with ChapterCheck CH19); live data is pulled
// from StoryFeed on a slow refresh clock, so no Start-order race can
// show a stale face. Programmatic UGUI, zero scene authoring (wired by
// SceneWire16 W18).
using System.Collections.Generic;
using System.Globalization;
using System.Text;
using UnityEngine;
using UnityEngine.UI;

namespace Sy.CrazyEstate
{
    public sealed class StoryFeedViewHost : MonoBehaviour
    {
        // Feed position law (R185): one canvas strictly under every wall
        // and desk face - the feed is an auxiliary seat, never the main
        // interface. The sidebar tab lives on the right edge of the 720
        // reference frame, away from every centered desk face.
        public const int CanvasOrder = 2;
        // right-edge anchor: the widest sidebar face (GateFace, 100px) ends
        // at 700 of the 720 reference frame - 300 = 360 - 50 - 10px safety
        // inset, so the docked seat never READS as edge-cropped (X1207 R2:
        // the flush 310 edge passed the in-frame law but reviewed as a
        // half-cropped card; was 318 = hard clip, X1192 P18 law)
        public const float SidebarEdgeX = 300f;
        public const float PageWidth = 700f;
        public const float PageHeight = 940f;
        public const float FaceRefreshSec = 0.5f;

        GameObject _panel;
        Button _tabButton;
        Text _gateFace;
        Text _latestFace;
        Text _albumTitleFace;
        Text _albumProgressFace;
        Text _timelineFace;
        Button _closeButton;
        float _faceClock;

        public int Taps;

        public bool PageOpen { get { return _panel != null && _panel.activeSelf; } }

        // ----- Pure decision layer (single truth, shared with ChapterCheck) -----

        // Copy key helpers: chapter titles reuse the T021-a almanac set

```

```

// verbatim, rare flips carry their own almanac names (1-based keys).
public static string ChapterTitleKey(int chapter)
{
    return "chapter_title_" + chapter.ToString("00", CultureInfo.InvariantCulture);
}

public static string FlipNameKey(int flipOrdinal)
{
    return "feed_flip_" + (flipOrdinal + 1).ToString("00", CultureInfo.InvariantCulture);
}

// Chapter-gate sidebar face: the live gate progress (chapters
// walked / total, moments lived / total) - honest zeros on the
// bare run, never invented.
public static string ComposeSidebarFace()
{
    return string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("feed_sidebar_face_fmt"),
        StoryFeed.AlbumUnlockedCount, StoryFeed.ChapterTotal,
        StoryFeed.CollectectedMoments, StoryFeed.MomentTotal);
}

// Sidebar latest-beat preview (U198 content axis, X1240): the
// sidebar carries the newest lived story beat verbatim (newest
// row first per the feed law) - or the waiting hint on the bare
// run. Reads the recorded stream only, never invents a beat.
public static string ComposeLatestFace()
{
    if (StoryFeed.RowCount == 0) return CopyTable.Get("feed_latest_none");
    return string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("feed_latest_fmt"), ComposeRowFace(StoryFeed.Rows()[0]));
}

// Album progress face: chapters collected, rare flips collected,
// moments lived - against the ring-constant totals.
public static string ComposeAlbumProgressFace()
{
    return string.Format(CultureInfo.InvariantCulture,
        CopyTable.Get("feed_album_progress_fmt"),
        StoryFeed.AlbumUnlockedCount, StoryFeed.ChapterTotal,
        StoryFeed.RareFlipCount, StoryFeed.RareFlipTotal,
        StoryFeed.CollectectedMoments, StoryFeed.MomentTotal);
}

// One timeline face line per recorded row: unlocks and gains carry
// the chapter number plus the almanac chapter title verbatim, rare
// flips carry the almanac flip name. The face only reads the row -
// it never re-derives or invents story facts.
public static string ComposeRowFace(StoryFeedRow row)
{

```

```

    var inv = CultureInfo.InvariantCulture;
    if (row.Kind == StoryFeedRowKind.ArcFlip)
    {
        var name = row.FlipOrdinal >= 0 && row.FlipOrdinal < ArcState.FlipCount
            ? CopyTable.Get(FlipNameKey(row.FlipOrdinal))
            : string.Empty;
        return string.Format(inv, CopyTable.Get("feed_row_flip_fmt"), name);
    }
    var title = row.Chapter >= 1 && row.Chapter <= PlayGate.ChapterCount
        ? CopyTable.Get(ChapterTitleKey(row.Chapter))
        : string.Empty;
    return string.Format(inv,
        CopyTable.Get(row.Kind == StoryFeedRowKind.ChapterUnlock
            ? "feed_row_chapter_fmt" : "feed_row_gain_fmt"),
        row.Chapter.ToString(inv), title);
}

// The story-page timeline body: one face line per row, newest
// first (StoryFeed.Rows is already newest-first), the empty-feed
// copy when nothing has happened yet.
public static string ComposeTimelineBody(List<StoryFeedRow> rows)
{
    if (rows == null || rows.Count == 0) return CopyTable.Get("feed_empty");
    var sb = new StringBuilder();
    for (var i = 0; i < rows.Count; i++)
    {
        if (i > 0) sb.Append('\n');
        sb.Append(ComposeRowFace(rows[i]));
    }
    return sb.ToString();
}

// Face fingerprint: sidebar + album + full timeline compose - the
// CH19 consumption-face double-run determinism check.
public static string FeedFaceFingerprint()
{
    var sb = new StringBuilder();
    sb.Append("s:").Append(ComposeSidebarFace()).Append(';');
    sb.Append("a:").Append(ComposeAlbumProgressFace()).Append(';');
    sb.Append("t:").Append(ComposeTimelineBody(StoryFeed.Rows)).Append(';');
    sb.Append("l:").Append(ComposeLatestFace());
    return sb.ToString();
}

// ----- Face wiring -----

void Start()
{
    BuildUi();
    SetPageOpen(false);
}

```

```

    }

    void Update()
    {
        _faceClock += Time.deltaTime;
        if (_faceClock < FaceRefreshSec) return;
        _faceClock = 0f;
        if (_gateFace != null) _gateFace.text = ComposeSidebarFace();
        if (_latestFace != null) _latestFace.text = ComposeLatestFace();
        if (PageOpen) RefreshPageFaces();
    }

    void SetPageOpen(bool open)
    {
        if (_panel != null) _panel.SetActive(open);
        if (open) RefreshPageFaces();
    }

    void RefreshPageFaces()
    {
        if (_albumProgressFace != null)
            _albumProgressFace.text = ComposeAlbumProgressFace();
        if (_timelineFace != null)
            _timelineFace.text = ComposeTimelineBody(StoryFeed.Rows());
    }

    void OnTabTapped()
    {
        Taps++;
        SetPageOpen(true);
    }

    void OnCloseTapped()
    {
        Taps++;
        SetPageOpen(false);
    }

    // ----- UI construction (programmatic, zero scene authoring) -----

    void BuildUi()
    {
        var canvasGo = new GameObject("StoryFeedCanvas", typeof(Canvas), typeof(CanvasScaler),
            typeof(GraphicRaycaster));
        var canvas = canvasGo.GetComponent<Canvas>();
        canvas.renderMode = RenderMode.ScreenSpaceOverlay;
        canvas.sortingOrder = CanvasOrder; // feed position law: under every wall/desk face
        var scaler = canvasGo.GetComponent<CanvasScaler>();
        // U184: ScaleWithScreenSize or the 720x1280 reference frame never
        // drives device scale (raw ConstantPixelSize = shrunken UI on phone).
    }

```

```

scaler.uiScaleMode = CanvasScaler.ScaleMode.ScaleWithScreenSize;
scaler.referenceResolution = new Vector2(720, 1280);

// Chapter-gate sidebar: right-edge tab + the live gate face.
_tabButton = MakeButton(canvasGo.transform, "StoryTab",
    new Vector2(SidebarEdgeX, 60f), 96, 76);
_tabButton.GetComponentInChildren<Text>().fontSize = 26;
_tabButton.GetComponentInChildren<Text>().text = CopyTable.Get("feed_sidebar_tab");
_tabButton.onClick.AddListener(OnTabTapped);

_gateFace = MakeText(canvasGo.transform, "GateFace", 16f,
    new Vector2(SidebarEdgeX, -18f), 100, 120);
_gateFace.raycastTarget = false; // passive strip under the tab
_gateFace.text = ComposeSidebarFace();

// A5 content fill (U198, X1240): the latest-beat preview strip
// under the gate face - the sidebar reads live before the page
// opens. Same right-edge column, wrapped beat lines read
// top-down, passive (never a tap thief).
_latestFace = MakeText(canvasGo.transform, "LatestFace", 14f,
    new Vector2(SidebarEdgeX, -160f), 100, 150);
_latestFace.raycastTarget = false;
_latestFace.alignment = TextAnchor.UpperLeft;
_latestFace.text = ComposeLatestFace();

// Story page (timeline + album), hidden until the tab tap.
_panel = new GameObject("StoryPage", typeof(RectTransform), typeof(Image));
_panel.transform.SetParent(canvasGo.transform, false);
var prt = (RectTransform)_panel.transform;
prt.sizeDelta = new Vector2(PageWidth, PageHeight);
prt.anchoredPosition = new Vector2(0f, -60f);
var img = _panel.GetComponent<Image>();
img.color = new Color(0.07f, 0.10f, 0.14f, 0.97f);

var titleFace = MakeText(_panel.transform, "StoryTitle", 40f,
    new Vector2(0f, 420f), 660, 64);
titleFace.text = CopyTable.Get("feed_page_title");

_albumTitleFace = MakeText(_panel.transform, "AlbumTitle", 24f,
    new Vector2(0f, 352f), 660, 44);
_albumTitleFace.text = CopyTable.Get("feed_album_title");

_albumProgressFace = MakeText(_panel.transform, "AlbumProgress", 21f,
    new Vector2(0f, 306f), 660, 42);
_albumProgressFace.raycastTarget = false;

_timelineFace = MakeText(_panel.transform, "TimelineFace", 19f,
    new Vector2(0f, -42f), 640, 690);
_timelineFace.alignment = TextAnchor.UpperLeft;
_timelineFace.raycastTarget = false; // reading face, never a tap thief

```

```

        _closeButton = MakeButton(_panel.transform, "StoryClose",
            new Vector2(0f, -430f), 220, 76);
        _closeButton.GetComponentInChildren<Text>().fontSize = 26;
        _closeButton.GetComponentInChildren<Text>().text = CopyTable.Get("feed_page_close");
        _closeButton.onClick.AddListener(OnCloseTapped);
    }

static Text MakeText(Transform parent, string name, float size, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var t = go.GetComponent<Text>();
    t.font = UIFont.Resolve();
    t.fontSize = Mathf.RoundToInt(size);
    t.alignment = TextAnchor.MiddleCenter;
    t.color = Color.white;
    return t;
}

static Button MakeButton(Transform parent, string name, Vector2 pos, int w, int h)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.sizeDelta = new Vector2(w, h);
    rt.anchoredPosition = pos;
    var img = go.GetComponent<Image>();
    img.color = new Color(0.16f, 0.24f, 0.32f, 1f);
    var btn = go.GetComponent<Button>();
    // One-Graphic-per-GO engine law: Tuanjie 2022.3 refuses a second
    // Graphic on the same GameObject (AddComponent<Text> returns null,
    // X1192 wave-1 proof), so the label rides a stretch-to-parent
    // child GO instead.
    var labelGo = new GameObject("Label", typeof(RectTransform), typeof(Text));
    labelGo.transform.SetParent(go.transform, false);
    var lrt = (RectTransform)labelGo.transform;
    lrt.anchorMin = Vector2.zero;
    lrt.anchorMax = Vector2.one;
    lrt.offsetMin = Vector2.zero;
    lrt.offsetMax = Vector2.zero;
    var label = labelGo.GetComponent<Text>();
    label.font = UIFont.Resolve();
    label.fontSize = 34;
    label.alignment = TextAnchor.MiddleCenter;
    label.color = Color.white;
    return btn;
}

```

```

    }
}
}
// ===== FILE: Assets\_Game\Scripts\Runtime\Story\StoryRuntime.cs =====
// G16 StoryRuntime hub (T022-d1a, M15 Oasis Story Almanac section 6): the
// single-writer mount of the three story rings - ONE PlayGate, ONE
// DramaBuff, ONE ArcState for the whole game. Hosts feed gameplay
// milestones THROUGH the hub (never straight to the rings), the hub
// translates the three evt anchors (evt_chapter_unlock, evt_drama_buff,
// evt_arc_flip) into the story event stream (the footprint-feed View
// slice reads this stream - story history is recorded, not re-derived),
// and it composes the ring-2/3 cut-window factor the bargain host
// applies per duel (the ch1 gift widens every duel window; Hike's
// dive-price comedy widens only his own while he still duels). Zero
// chapters = the bare baseline everywhere - a gain is comfort, never a
// gate (U091). Pure decision layer like the rings: zero RNG, zero clocks.
using System;
using System.Collections.Generic;
using System.Text;

namespace Sy.CrazyEstate
{
    /// <summary>One story event in the stream: which anchor fired, in what
    /// order, carrying which chapter / gain / flip ordinal.</summary>
    public sealed class StoryEventRecord
    {
        public string EventId;
        public int Seq; // stream order, 1-based, strictly monotone
        public int Payload; // chapter number / gifting chapter / flip ordinal
    }

    public static class StoryRuntime
    {
        static PlayGate _gate = new PlayGate();
        static DramaBuff _buff = new DramaBuff();
        static ArcState _arc = new ArcState();
        static readonly List<StoryEventRecord> _stream = new List<StoryEventRecord>();
        static int _seq;

        static StoryRuntime()
        {
            Wire();
        }

        // ---- ring-1/2/3 single-writer mounts (hosts read, never replace) ----

        public static PlayGate Gate { get { return _gate; } }
        public static DramaBuff Buff { get { return _buff; } }
        public static ArcState Arc { get { return _arc; } }
    }
}

```

```

/// <summary>The story event stream (evt anchor translations, append
/// order). The footprint-feed View slice is the consumer.</summary>
public static IReadOnlyList<StoryEventRecord> Stream { get { return _stream; } }

// ---- host feed API ----

/// <summary>Feed one gameplay milestone: the ring-1 gate latches the
/// footprint and walks, then the ring-2 gains and ring-3 flips sync
/// to the walked gate (both idempotent latches inside). Returns the
/// highest chapter THIS feed unlocked (0 = the armed gate waits -
/// zero dead ends, keep playing the core loop).</summary>
public static int Feed(GateFeed feed, double payload = 0.0, int intPayload = 0)
{
    var unlocked = _gate.Feed(feed, payload, intPayload);
    _buff.SyncFromGate(_gate);
    _arc.SyncFromGate(_gate);
    return unlocked;
}

/// <summary>Closed-deal feed (the bargain host's receipt.Accepted
/// point): the ring-1 first-deal footprint plus the ring-3 viewer
/// counter in one call - the livestream number reads REAL closed
/// deals, never invented. listingId &lt; 0 = the unbound guided boot
/// duel, which is still one real closed deal. The H07 future-city
/// pre-sale latches the ch8 footprint at the same receipt (one deal
/// position, two footprint facts - the id space is ListingTable).</summary>
public static void FeedDealClosed(int listingId)
{
    Feed(GateFeed.DealClosed, 0.0, listingId);
    if (listingId == PlayGate.FutureCityListingId) Feed(GateFeed.HoldingH07);
    _arc.FeedDealClosed();
}

/// <summary>Duel-start feed (the bargain host's listing-bind point):
/// only Hike duels count toward the ch11 "only you treated me as a
/// peer" verdict - the warmth is earned by haggling HIM.</summary>
public static void FeedDuelStart(int agentId)
{
    if (agentId == ArcState.AgentHike) _arc.FeedHikeDuel();
}

// ---- ring-2/3 bargain-window composition (host re-computation) ----

/// <summary>Composite cut-window factor the bargain host sets on the
/// sim per duel: the ring-2 ch1 gift widens EVERY duel window; the
/// ring-3 Hike dive-price comedy widens only HIS window while he
/// still duels (after the ch11 noodle exit the factor returns to the
/// gift-only face). Zero chapters = 1.0 identity (bare baseline).</summary>
public static double ComposeCutWindowFactor(int agentId)
{

```

```

        var factor = 1.0 + _buff.BargainWindowBonusPct() / 100.0;
        if (agentId == ArcState.AgentHike) factor *= _arc.HikeCutWindowFactor();
        return factor;
    }

    /// <summary>Hub fingerprint: the three ring fingerprints plus the
    /// stream trace - the CH15 double-run determinism face.</summary>
    public static string HubFingerprint()
    {
        var sb = new StringBuilder();
        sb.Append(_gate.Fingerprint()).Append('#');
        sb.Append(_buff.Fingerprint()).Append('#');
        sb.Append(_arc.Fingerprint()).Append('#');
        foreach (var e in _stream)
            sb.Append(e.Seq).Append(':').Append(e.EventId).Append(':').Append(e.Payload).Append(';');
        return sb.ToString();
    }

    // ---- test seam (Editor checks only; production never resets) ----

    /// <summary>Fresh hub for the Editor check law (CH15): recreates the
    /// three rings, rewires the anchor translations and clears the
    /// stream. Production code must never call this.</summary>
    public static void ResetForChecks()
    {
        _gate = new PlayGate();
        _buff = new DramaBuff();
        _arc = new ArcState();
        _stream.Clear();
        _seq = 0;
        Wire();
    }

    static void Wire()
    {
        _gate.OnChapterUnlocked += ch => Append(PlayGate.ChapterUnlockEventId, ch);
        _buff.OnGainMounted += ch => Append(DramaBuff.GainMountEventId, ch);
        _arc.OnArcFlip += f => Append(ArcState.ArcFlipEventId, f);
    }

    static void Append(string eventId, int payload)
    {
        _seq++;
        _stream.Add(new StoryEventRecord { EventId = eventId, Seq = _seq, Payload = payload });
    }
}

```